ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Đồ án chuyên ngành

Báo cáo

Trục tích hợp dữ liệu (Data Integration Hub)

Giảng viên hướng dẫn:	ThS. Nguyễn Cao Đạt	
	ThS. Nguyễn Quang Sang	
Sinh viên thực hiện:	Trần Mạnh Tuấn	2213807
	Phạm Lê Huy	2252260

HỒ CHÍ MINH, THÁNG 12 2025

Mục lục

Danh mục ký hiệu và từ viết tắt **8**

1 Giới thiệu **9**

1.1 Bối cảnh và lý do chọn đề tài 9

1.2 Mục tiêu nghiên cứu 10

1.3 Phạm vi nghiên cứu 11

1.4 Phương pháp nghiên cứu 11

1.5 Đóng góp của luận văn 12

2 Tổng quan lý thuyết và nghiên cứu liên quan **13**

2.1 Các khái niệm cơ bản 13

2.1.1 Data Integration Hub 13

2.1.2 OLTP và OLAP 15

2.1.2.1 OLTP (Online Transaction Processing) 15

2.1.2.2 OLAP (Online Analytical Processing) 16

2.1.2.3 So sánh OLTP và OLAP 16

2.1.3 ETL và ELT 17

2.1.3.1 Quy trình ETL (Extract - Transform - Load) 17

2.1.3.2 Quy trình ELT (Extract - Load - Transform) 18

2.1.3.3 Ứng dụng trong bối cảnh trường đại học 19

2.2 Các kiến trúc tích hợp dữ liệu 20

2.2.1 Kiến trúc điểm-nối-điểm (Point-to-Point - P2P) 20

2.2.2 Kiến trúc trục và nan hoa (Hub-and-Spoke) 21

2.2.3 Kiến trúc bus dịch vụ doanh nghiệp (Enterprise Service Bus - ESB) 21

2.2.4 Kiến trúc kết nối dựa trên API (API-led Connectivity) 22

2.2.5 So sánh và lựa chọn kiến trúc cho bối cảnh đại học 23

2.2.5.1 Những hạn chế của kiến trúc truyền thống 24

2.2.5.2 Lý do lựa chọn kiến trúc API-led Connectivity 24

3 Thiết kế kiến trúc 26

3.1 Yêu cầu chức năng và phi chức năng 26
3.1.1 Yêu cầu chức năng (Functional Requirements) 26
3.1.2 Yêu cầu phi chức năng (Non-functional Requirements) 27
3.2 Sơ đồ kiến trúc tổng thể 27
3.3 Giải thích chi tiết các thành phần 27
3.3.1 Các hệ thống nguồn (Data Sources) 28
3.3.2 Trục tích hợp thông tin (Information Integration Hub) 28
3.3.2.1 Máy chủ Tích hợp Dữ liệu (Data Integration Server) . . . 28
3.3.2.2 Máy chủ Chia sẻ Dữ liệu (Data Sharing Server) 29
3.3.2.3 Kho dữ liệu trung tâm (Central Data Store) 29
3.3.3 Cổng API (API Gateway) 29
3.3.4 Thiết kế chi tiết Module Quản lý Hệ thống 31
3.3.4.1 Luồng Xác thực (Flow) 31
3.3.4.2 Luồng Phân quyền (Authorization Flow) 31
3.3.4.3 Cấu trúc Cơ sở dữ liệu cho RBAC 34

4 Các công nghệ sử dụng trong hệ thống 35

4.1 Nền tảng Backend: NestJS 35
4.2 ORM & Database Migration Tool cho NestJS: Prisma 37
4.3 Cơ sở dữ liệu trung tâm: PostgreSQL 38
4.4 Giao diện Quản trị (System Management Dashboard): Next.js & TailwindCSS 39
4.5 Kong API Gateway 40

5 Giám sát, Ghi log và Cảnh báo hệ thống 42

5.1 Kiến trúc Observability tổng quan 42
5.2 Chiến lược ghi log (Logging Strategy) 43
5.2.1 Structured Logging (Log có cấu trúc) 43
5.2.2 Distributed Tracing với Correlation ID 44
5.3 Các thành phần thu thập chi tiết 44
5.3.1 Kong API Gateway Logs 44
5.3.2 Application Logs (NestJS) 44
5.3.3 Database & System Logs 44
5.4 Dashboard giám sát và Cảnh báo 44
5.4.1 Dashboard (Kibana) 44

5.4.2 Cơ chế Cảnh báo (Alerting) 45

6 Data Governance và Quy trình Tích hợp liên tục (CI/CD) 46

6.1 Quản trị Dữ liệu (Data Governance) và Chất lượng Dữ liệu (Data Quality) 46

6.1.1 Quản trị Dữ liệu (Data Governance) 46

6.1.1.1 Quản lý Siêu dữ liệu (Metadata Management) 46

6.1.1.2 Quyền sở hữu dữ liệu (Data Ownership & Stewardship) . 46

6.1.1.3 Kiểm soát truy cập (Data Access Control) 47

6.1.2 Kiểm soát Chất lượng Dữ liệu (Data Quality - DQ) 47

6.1.3 Cơ chế xử lý lỗi dữ liệu 47

6.2 Tích hợp Data Governance vào CI/CD 47

6.2.1 Quy trình triển khai 48

7 Chiến lược Sao lưu và Phục hồi Dữ liệu (Backup Strategy) 50

7.1 Chiến lược và Chu kỳ sao lưu 51

7.2 Phương thức kỹ thuật 51

7.3 Cam kết chỉ số phục hồi (RPO & RTO) 51

8 Kế hoạch thực hiện đồ án tốt nghiệp 52

8.1 Mục tiêu tổng thể 52

8.2 Phân chia giai đoạn thực hiện 52

8.2.1 Giai đoạn 1: Phân tích & Hoàn thiện thiết kế (01/2026) 52

8.2.2 Giai đoạn 2: Phát triển Core Services (02/2026) 53

8.2.3 Giai đoạn 3: Xây dựng hạ tầng & DevOps (03/2026) 54

8.2.4 Giai đoạn 4: Hoàn thiện bảo mật, logging & monitoring (04/2026) . 55

8.2.5 Giai đoạn 5: Kiểm thử tổng thể & tối ưu (05/2026 - đầu tháng) . . 55

8.2.6 Giai đoạn 6: Hoàn thiện báo cáo & bảo vệ (Cuối 05/2026) 56

9 Tổng kết đồ án chuyên ngành 57

9.1 Những điểm đã đạt được 57

9.2 Những điểm cần cải thiện 58

9.3 Kết luận chung 58

Danh sách hình vẽ

2.1.1 Mô hình quy trình ETL điển hình, trích xuất dữ liệu từ các hệ thống nguồn (OLTP) để nạp vào Kho dữ liệu (Data Warehouse)[13] 18
2.2.1 Kiến trúc Point-to-point[11] 20
2.2.2 Kiến trúc Enterprise Service Bus[12] 22
2.2.3 Minh hoạ cho kiến trúc API-led Connectivity[10] 23

3.2.1 Sơ đồ kiến trúc tổng thể của Trục tích hợp dữ liệu 28
3.3.1 Login Flow cho hệ thống 30
3.3.2 Luồng xử lý Xác thực của hệ thống 32
3.3.3 Luồng xử lý refresh token của hệ thống 33
3.3.4 RBAC 34

5.1.1 Kiến trúc dòng chảy dữ liệu giám sát (Log Flow) 43

6.2.1 Quy trình CI/CD tích hợp kiểm tra dữ liệu và giám sát 49

Danh sách bảng

2.1.1 So sánh sự khác biệt giữa OLTP và OLAP 17
2.1.2 So sánh giữa quy trình ETL và ELT 19

4.1.1 So sánh các backend framework 36
4.2.1 So sánh các ORM phổ biến trong hệ sinh thái Nest.js 38
4.3.1 So sánh các hệ quản trị cơ sở dữ liệu 39
4.5.1 So sánh các API Gateway phổ biến 41

5.4.1 Ma trận quy tắc cảnh báo 45

7.0.1 So sánh MinIO với các giải pháp lưu trữ đám mây 50

Danh mục ký hiệu và từ viết tắt

API	Application Programming Interface (Giao diện lập trình ứng dụng)
AWS	Amazon Web Services
BI	Business Intelligence (Trí tuệ doanh nghiệp)
CI/CD	Continuous Integration / Continuous Deployment
CSDL	Cơ sở dữ liệu
ELK	Elasticsearch, Logstash, Kibana
ESB	Enterprise Service Bus (Trục dịch vụ doanh nghiệp)
HRM	Human Resource Management (Hệ thống quản lý nhân sự)
JWT	JSON Web Token
LMS	Learning Management System (Hệ thống quản lý học tập)
MVP	Minimum Viable Product (Sản phẩm khả thi tối thiểu)
ODS	Operational Data Store (Kho dữ liệu hoạt động)
OLAP	Online Analytical Processing (Xử lý phân tích trực tuyến)
OLTP	Online Transaction Processing (Xử lý giao dịch trực tuyến)
P2P	Point-to-Point (Mô hình điểm - điểm)
RBAC	Role-Based Access Control (Kiểm soát truy cập dựa trên vai trò)
SIS	Student Information System (Hệ thống thông tin sinh viên)

Chương 1

Giới thiệu

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh cuộc Cách mạng Công nghiệp 4.0, chuyển đổi số đang trở thành xu thế tất yếu và là yêu cầu cấp thiết đối với các cơ sở giáo dục đại học. Một trường đại học hiện đại không chỉ là nơi đào tạo và nghiên cứu mà còn là một tổ chức vận hành phức tạp với hàng chục ngàn sinh viên, giảng viên và nhân viên. Để quản lý bộ máy này, nhà trường thường sử dụng nhiều hệ thống thông tin chuyên biệt như: Hệ thống quản lý đào tạo (SIS), Hệ thống quản lý nhân sự (HRM), Hệ thống quản lý tài chính - kế toán, Hệ thống quản lý thư viện và các hệ thống E-learning (LMS).

Tuy nhiên, thực trạng phổ biến tại nhiều trường đại học hiện nay là sự phân mảnh dữ liệu (Data Silos). Các phòng ban chức năng thường vận hành các hệ thống phần mềm riêng lẻ, được phát triển bởi các nhà cung cấp khác nhau hoặc vào các thời điểm khác nhau. Điều này dẫn đến những vấn đề nghiêm trọng:

- **Thiếu tính nhất quán dữ liệu:** Thông tin của cùng một sinh viên (ví dụ: trạng thái đóng học phí) có thể sai lệch giữa Phòng Tài chính và Phòng Đào tạo, gây khó khăn cho việc đăng ký môn học hoặc xét tốt nghiệp.

- **Khó khăn trong tích hợp và chia sẻ:** Việc kết nối giữa các hệ thống thường được thực hiện theo mô hình điểm - điểm (Point-to-Point). Khi số lượng hệ thống tăng lên, mạng lưới kết nối trở nên chằng chịt ("Spaghetti Architecture"), khó bảo trì và mở rộng.

- **Trải nghiệm người dùng kém:** Sinh viên và giảng viên phải truy cập nhiều cổng thông tin khác nhau để tìm kiếm dữ liệu liên quan đến bản thân.

- **Khó khăn trong báo cáo và ra quyết định:** Lãnh đạo nhà trường thiếu một cái nhìn toàn cảnh (360-degree view) về hoạt động của trường do dữ liệu nằm rải rác, không được tổng hợp kịp thời.

Trước nhu cầu cấp thiết về việc đồng bộ hóa dữ liệu nội bộ và kết nối với các hệ thống bên ngoài (như CSDL Giáo dục Quốc gia - HEMIS, ĐHQG-HCM), đề tài **"Tìm hiểu và hiện thực hệ thống trục tích hợp dữ liệu cho trường đại học"** được lựa chọn thực hiện. Đề tài tập trung giải quyết bài toán phân mảnh dữ liệu bằng cách xây dựng một trục tích hợp trung tâm (Data Integration Hub) hiện đại, đảm bảo tính chính xác, bảo mật và sẵn sàng của dòng chảy dữ liệu trong môi trường giáo dục.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống trục tích hợp dữ liệu có khả năng thu thập, chuẩn hóa, lưu trữ và chia sẻ dữ liệu giữa các hệ thống thông tin trong trường đại học một cách tự động và nhất quán.

Các mục tiêu cụ thể bao gồm:

1. **Nghiên cứu lý thuyết:** Tìm hiểu các kiến trúc tích hợp dữ liệu phổ biến (Hub-and-Spoke, ESB) và đặc biệt là kiến trúc hướng API (API-led Connectivity). Nghiên cứu các kỹ thuật về quản trị dữ liệu (Data Governance), đảm bảo chất lượng dữ liệu (Data Quality) và bảo mật hệ thống.

2. **Thiết kế kiến trúc hệ thống:** Đề xuất một kiến trúc trục tích hợp dữ liệu tối ưu cho ngữ cảnh trường đại học, khắc phục được nhược điểm của các mô hình cũ, đảm bảo tính linh hoạt, khả năng mở rộng (Scalability) và dễ dàng bảo trì.

3. **Hiện thực hóa giải pháp:** Xây dựng trục tích hợp dữ liệu (MVP - Minimum Viable Product) với các thành phần cốt lõi:

   - *Data Integration Server:* Thu thập và chuẩn hóa dữ liệu từ các nguồn khác nhau.
   - *Data Sharing Server:* Cung cấp API chuẩn hóa cho các ứng dụng khai thác (Mobile App, Portal).
   - *Centralized Database:* Kho dữ liệu hoạt động (ODS) lưu trữ dữ liệu nhất quán.
   - *Các thành phần hỗ trợ:* Định thời (Scheduler), Bảo mật (API Gateway), và Giám sát (Monitoring).

4. **Đánh giá và kiểm thử:** Kiểm chứng khả năng hoạt động, hiệu năng và tính đúng đắn của dữ liệu sau khi tích hợp.

1.3 Phạm vi nghiên cứu

Để đảm bảo tính khả thi trong khuôn khổ thời gian của đồ án, phạm vi nghiên cứu được giới hạn như sau:

- **Về dữ liệu:** Tập trung vào các nhóm dữ liệu cốt lõi trong trường đại học bao gồm: Dữ liệu Sinh viên, Dữ liệu Giảng viên/Viên chức, Dữ liệu Đào tạo (Môn học, Điểm số) và Dữ liệu Tài chính (Học phí cơ bản).
- **Về hệ thống nguồn:** Mô phỏng các hệ thống nguồn (Source Systems) đại diện cho các phòng ban (Phòng Đào tạo, Phòng Công tác Sinh viên, Phòng Tài chính) thông qua các cơ sở dữ liệu giả lập có cấu trúc khác nhau.
- **Về công nghệ:** Sử dụng các công nghệ mã nguồn mở hiện đại phù hợp với kiến trúc Microservices và Containerization (NestJS, Docker, PostgreSQL, MinIO, ELK Stack).
- **Giới hạn:** Đề tài tập trung vào phần "Backend/Middleware"(Trục tích hợp). Các ứng dụng người dùng cuối (như Mobile App hay Web Portal) chỉ được xây dựng ở mức độ demo cơ bản để minh họa cho việc gọi API từ trục tích hợp.

1.4 Phương pháp nghiên cứu

Đề tài áp dụng kết hợp các phương pháp nghiên cứu sau:

- **Phương pháp nghiên cứu tài liệu:** Tổng hợp lý thuyết từ các sách chuyên khảo, bài báo khoa học và tài liệu kỹ thuật về kiến trúc phần mềm, Data Integration, Enterprise Service Bus (ESB) và API Gateway.
- **Phương pháp phân tích và thiết kế hệ thống:** Áp dụng tư duy thiết kế hướng miền (Domain-Driven Design) để phân tích nghiệp vụ trường đại học. Sử dụng ngôn ngữ UML để mô hình hóa kiến trúc và luồng dữ liệu.
- **Phương pháp thực nghiệm:**
  - Xây dựng môi trường giả lập để triển khai hệ thống.

  - Áp dụng quy trình phát triển phần mềm linh hoạt (Agile/Scrum).
  - Sử dụng CI/CD để tự động hóa quy trình kiểm thử và triển khai.

- **Phương pháp so sánh, đánh giá:** So sánh kiến trúc đề xuất với các mô hình truyền thống để làm rõ ưu/nhược điểm.

1.5 Đóng góp của luận văn

Luận văn mang lại những đóng góp chính về cả mặt lý thuyết và thực tiễn:

- **Về mặt kiến trúc:** Đề xuất một mô hình kiến trúc tham chiếu cho trục tích hợp dữ liệu trường đại học dựa trên tư duy *API-led Connectivity*. Kiến trúc này phân tách rõ ràng giữa lớp Hệ thống (System), lớp Quy trình (Process) và lớp Trải nghiệm (Experience), giúp tăng tính linh hoạt và khả năng tái sử dụng dịch vụ.

- **Về mặt kỹ thuật:** Cung cấp một bộ giải pháp kỹ thuật cụ thể (Technical Stack) và quy trình vận hành (DevOps, DataOps) để hiện thực hóa trục tích hợp, bao gồm các cơ chế nâng cao như lập lịch động (Dynamic Scheduler), sao lưu và phục hồi điểm thời gian (Point-in-Time Recovery), và bảo mật đa lớp.

- **Về mặt ứng dụng:** Sản phẩm của luận văn có thể được sử dụng làm nền tảng cơ sở (Baseline) để triển khai thực tế tại trường, giúp giải quyết bài toán đồng bộ dữ liệu, làm tiền đề cho các hệ thống báo cáo thông minh (BI) và ứng dụng di động cho sinh viên (Student Super App).

Chương 2

Tổng quan lý thuyết và nghiên cứu liên quan

2.1 Các khái niệm cơ bản

2.1.1 Data Integration Hub

Trong bối cảnh công nghệ số hiện đại, các tổ chức, doanh nghiệp phải đối mặt với thách thức quản lý và xử lý lượng dữ liệu khổng lồ từ nhiều nguồn phân tán. Sự phức tạp này đòi hỏi một giải pháp kiến trúc có khả năng tập trung hóa và điều phối luồng dữ liệu một cách hiệu quả. Nền tảng đảm nhận nhiệm vụ này chính là Trục tích hợp dữ liệu (Data Integration Hub).

Trục tích hợp dữ liệu hoạt động như một điểm trung gian, kết nối các hệ thống nguồn (data sources) và các ứng dụng tiêu thụ dữ liệu (data consumers). Thay vì tạo ra các kết nối điểm-nối-điểm (point-to-point) phức tạp, trục tích hợp dữ liệu đơn giản hóa kiến trúc bằng cách cung cấp một điểm trao đổi dữ liệu duy nhất, giúp việc lưu trữ, xử lý và đồng bộ hóa thông tin trở nên dễ dàng và nhất quán hơn. Trong môi trường đại học, nơi dữ liệu sinh viên, giảng viên, tài chính, và đào tạo phân tán ở nhiều phòng ban, việc áp dụng một trục tích hợp dữ liệu càng trở nên cấp thiết để đảm bảo thông tin được xuyên suốt và chính xác.

Theo định nghĩa của Informatica, một trong những công ty hàng đầu về quản lý dữ liệu:

> "Trung tâm tích hợp dữ liệu đóng vai trò là nền tảng tập trung, cho phép các tổ chức tích hợp, quản lý và quản trị dữ liệu một cách liền mạch trên nhiều hệ thống và ứng dụng phân tích khác nhau. Hoạt động như một kênh dẫn dữ

> liệu, trung tâm tích hợp dữ liệu hợp nhất thông tin từ các nguồn khác nhau, chẳng hạn như cơ sở dữ liệu, dịch vụ đám mây và hệ thống cũ, tạo điều kiện thuận lợi cho việc trao đổi và đồng bộ hóa dữ liệu hiệu quả. Trung tâm này hoạt động như một đơn vị trung gian, đảm bảo dữ liệu được truyền tải thông suốt giữa các hệ thống, đồng thời cung cấp các khả năng về chất lượng dữ liệu, bảo mật và quản trị." [6]

Trong khi đó, Amazon Web Services (AWS) định nghĩa khái niệm "Tích hợp dữ liệu"ở phạm vi rộng hơn, nhấn mạnh đến mục tiêu cuối cùng là tạo ra giá trị từ dữ liệu:

> "Tích hợp dữ liệu là quá trình đạt được quyền truy cập và phân phối nhất quán cho tất cả các loại dữ liệu trong doanh nghiệp. [...] Tích hợp dữ liệu liên quan đến các kỹ thuật, công cụ và phương thức về kiến trúc nhằm hợp nhất các nguồn dữ liệu khác nhau cho hoạt động phân tích. Do đó, các tổ chức có thể xem đầy đủ dữ liệu của họ để biết thông tin chuyên sâu và nghiệp vụ thông minh có giá trị cao." [1]

Từ hai định nghĩa trên, có thể thấy Trục tích hợp dữ liệu là một kiến trúc cụ thể để hiện thực hóa việc tích hợp dữ liệu. Nó không chỉ là công cụ truyền tải mà còn là một nền tảng quản trị tập trung. Các nhiệm vụ chính của một trục tích hợp dữ liệu bao gồm:

- **Thu thập và phân phối dữ liệu:** Tiếp nhận dữ liệu từ các hệ thống nguồn và phân phối đến các hệ thống đích theo mô hình publish/subscribe (đăng ký/xuất bản) [2;8].
- **Điều phối và định tuyến:** Tổng hợp và chuyển tiếp dữ liệu đến các hệ thống hoặc ứng dụng cần thiết, đồng thời áp dụng các quy tắc phân quyền để đảm bảo an toàn thông tin.
- **Đảm bảo chất lượng dữ liệu:** Kiểm tra tính hợp lệ, nhất quán và tuân thủ các tiêu chuẩn định sẵn của dữ liệu. Dữ liệu không đạt chuẩn có thể bị từ chối hoặc chuyển vào một quy trình làm sạch riêng, nhằm duy trì sự toàn vẹn cho toàn hệ thống.
- **Giám sát và ghi nhật ký (Logging):** Theo dõi toàn bộ quá trình trao đổi dữ liệu, ghi lại các sự kiện quan trọng để phục vụ cho việc kiểm tra, gỡ lỗi và phân tích hiệu suất.
- **Lập lịch (Scheduling):** Tự động hóa việc thực thi các tác vụ tích hợp dữ liệu theo những khoảng thời gian được định trước.

Việc triển khai một trục tích hợp dữ liệu mang lại nhiều vai trò và lợi ích quan trọng, đặc biệt với các hệ thống phức tạp, đa nguồn dữ liệu:

- **Giảm sự phức tạp của hệ thống:** Bằng cách thay thế mạng lưới kết nối point-to-point chằng chịt, hệ thống trở nên dễ quản lý, bảo trì và phát triển hơn.
- **Tăng cường tính nhất quán của dữ liệu:** Đảm bảo rằng tất cả các ứng dụng đều truy cập vào cùng một nguồn dữ liệu đã được xác thực và đồng bộ, loại bỏ tình trạng xung đột và sai lệch thông tin[3;8].
- **Tăng tính linh hoạt và khả năng mở rộng:** Khi một hệ thống mới cần tham gia vào mạng lưới, chỉ cần kết nối nó với trục trung tâm thay vì phải xây dựng kết nối riêng lẻ tới tất cả các hệ thống khác[2].
- **Nền tảng cho phân tích dữ liệu:** Trục tích hợp dữ liệu cung cấp dữ liệu sạch, nhất quán và tập trung, tạo điều kiện lý tưởng cho các hoạt động phân tích kinh doanh (Business Intelligence), học máy (Machine Learning) và hỗ trợ ra quyết định[5;6].

Tóm lại, trục tích hợp dữ liệu là một thành phần kiến trúc chiến lược, đóng vai trò trung gian điều phối luồng thông tin giữa các nguồn dữ liệu và các hệ thống tiêu thụ, qua đó đảm bảo sự ổn định, nhất quán và linh hoạt cho toàn bộ hệ sinh thái dữ liệu của một tổ chức.

2.1.2 OLTP và OLAP

Trong lĩnh vực xử lý dữ liệu, hai loại hệ thống chính thường được nhắc đến là OLTP và OLAP, phục vụ cho hai mục đích hoàn toàn khác nhau.

2.1.2.1 OLTP (Online Transaction Processing)

OLTP, viết tắt của Xử lý giao dịch trực tuyến (Online Transaction Processing), ám chỉ các hệ thống được thiết kế để quản lý và xử lý một khối lượng lớn các giao dịch ngắn, diễn ra trong thời gian thực. Các giao dịch này thường là các thao tác cơ sở dữ liệu cơ bản như thêm (INSERT), sửa (UPDATE), và xóa (DELETE).

Ví dụ điển hình là hệ thống của máy ATM, hệ thống đặt vé máy bay, hay hệ thống quản lý sinh viên của một trường đại học khi sinh viên đăng ký môn học. Mục tiêu chính của hệ thống OLTP là đảm bảo tính sẵn sàng cao, tốc độ xử lý nhanh, khả năng chịu tải đồng thời lớn và tính toàn vẹn dữ liệu (thông qua các thuộc tính ACID - Atomicity, Consistency, Isolation, Durability)[7]. Do đó, cơ sở dữ liệu của OLTP thường được thiết

kế theo mô hình quan hệ và được chuẩn hóa cao để tối ưu cho các thao tác ghi và cập nhật.

2.1.2.2 OLAP (Online Analytical Processing)

Ngược lại với OLTP, OLAP (Xử lý phân tích trực tuyến - Online Analytical Processing) là các hệ thống được tối ưu cho việc thực hiện các truy vấn phân tích phức tạp trên một lượng lớn dữ liệu. Thay vì xử lý các giao dịch hàng ngày, mục tiêu của OLAP là hỗ trợ việc ra quyết định kinh doanh, phân tích xu hướng và khai phá dữ liệu[9].

Hệ thống OLAP thường hoạt động trên các Kho dữ liệu (Data Warehouse), chứa dữ liệu lịch sử được tổng hợp từ nhiều hệ thống OLTP khác nhau. Nền tảng của OLAP là mô hình dữ liệu đa chiều, thường được hình dung như một khối dữ liệu (Data Cube). Theo Jiawei Han và các cộng sự, mô hình này cho phép dữ liệu được tổ chức theo nhiều chiều (dimensions) và mỗi chiều lại chứa nhiều cấp độ trừu tượng khác nhau (concept hierarchies), giúp người dùng có thể linh hoạt xem xét dữ liệu từ nhiều góc độ[4].

Các thao tác phân tích điển hình trên một khối dữ liệu OLAP bao gồm[4]:

- **Roll-up (Tổng hợp):** Thao tác tổng hợp dữ liệu bằng cách đi lên các cấp cao hơn trong một hệ thống phân cấp của chiều dữ liệu.
- **Drill-down (Chi tiết hóa):** Thao tác ngược lại với roll-up, đi từ dữ liệu ít chi tiết đến dữ liệu chi tiết hơn.
- **Slice (Cắt lát):** Chọn một giá trị trên một chiều để tạo ra một khối dữ liệu con.
- **Dice (Cắt khối):** Chọn một dải giá trị trên hai hoặc nhiều chiều để tạo ra một khối dữ liệu con.

2.1.2.3 So sánh OLTP và OLAP

Sự khác biệt cơ bản giữa hai hệ thống có thể được tóm tắt trong bảng dưới đây.

Trong bối cảnh của một trường đại học, hệ thống quản lý sinh viên (nơi sinh viên đăng ký môn học, phòng đào tạo nhập điểm) là một hệ thống OLTP điển hình. Trong khi đó, một hệ thống báo cáo cho ban giám hiệu để phân tích xu hướng tuyển sinh qua các năm, tỷ lệ tốt nghiệp theo từng khoa, hoặc hiệu quả sử dụng phòng học sẽ là một hệ thống OLAP, lấy dữ liệu từ hệ thống quản lý sinh viên và các nguồn khác.

Tiêu chí	OLTP	OLAP
Mục tiêu chính	Quản lý các hoạt động nghiệp vụ hàng ngày	Hỗ trợ ra quyết định và phân tích chiến lược
Loại thao tác	Các giao dịch nhỏ, nhanh (INSERT, UPDATE, DELETE)	Các truy vấn phức tạp, tổng hợp (SELECT, GROUP BY)
Tối ưu cho	Thao tác ghi (Write-intensive)	Thao tác đọc (Read-intensive)
Nguồn dữ liệu	Dữ liệu hoạt động, thời gian thực	Dữ liệu lịch sử, được tổng hợp từ nhiều nguồn
Cấu trúc CSDL	Chuẩn hóa cao (Normalized - ER Model)	Phi chuẩn hóa (Denormalized - Star/Snowflake Schema)
Đối tượng sử dụng	Nhân viên nghiệp vụ, người dùng cuối	Nhà phân tích dữ liệu, nhà quản lý

Bảng 2.1.1: So sánh sự khác biệt giữa OLTP và OLAP

2.1.3 ETL và ELT

Khi dữ liệu được thu thập từ nhiều hệ thống nguồn (như các hệ thống OLTP), chúng thường ở dạng thô, không đồng nhất về cấu trúc và định dạng. Để có thể sử dụng dữ liệu này cho mục đích phân tích trong một kho dữ liệu (Data Warehouse), cần có một quy trình để xử lý và hợp nhất chúng. ETL và ELT là hai quy trình phổ biến nhất để thực hiện nhiệm vụ này.

2.1.3.1 Quy trình ETL (Extract - Transform - Load)

ETL là viết tắt của **Extract (Trích xuất), Transform (Biến đổi), Load (Nạp)**. Đây là một quy trình truyền thống và được sử dụng rộng rãi, bao gồm ba giai đoạn chính[13]:

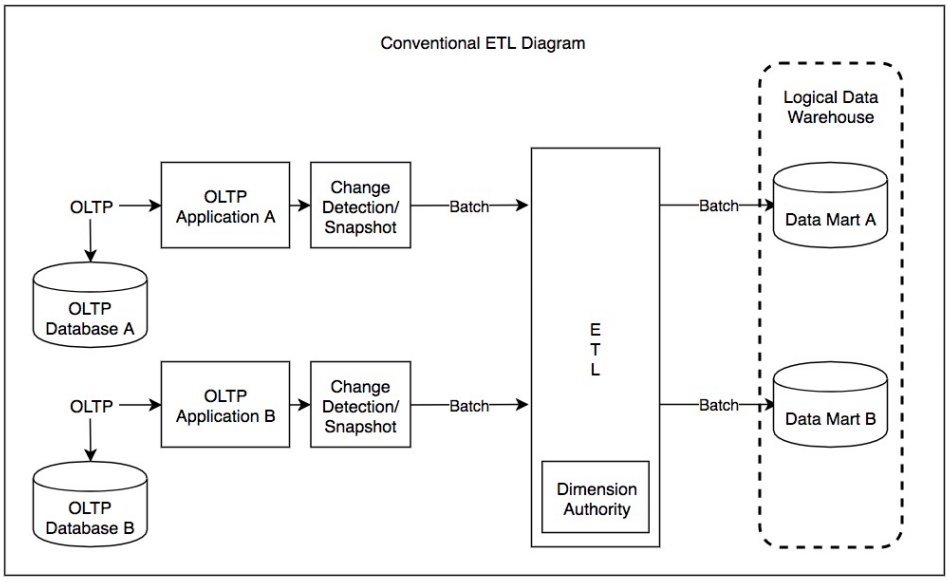
1. **Extract (Trích xuất):** Dữ liệu được trích xuất từ các hệ thống nguồn đa dạng như cơ sở dữ liệu quan hệ (SQL), cơ sở dữ liệu NoSQL, các file phẳng (CSV, XML, JSON), hoặc các API. Ở giai đoạn này, dữ liệu được lấy ra khỏi nguồn và đưa vào một khu vực tạm gọi là Staging Area.

2. **Transform (Biến đổi):** Đây là giai đoạn quan trọng nhất, nơi dữ liệu thô được xử lý để đảm bảo chất lượng và tính nhất quán. Các tác vụ biến đổi phổ biến bao gồm:

   - **Làm sạch (Cleansing):** Sửa lỗi chính tả, xử lý các giá trị bị thiếu (null), loại bỏ dữ liệu trùng lặp.

- **Chuẩn hóa (Standardization):** Đưa các trường dữ liệu về một định dạng chung. Ví dụ, chuyển đổi trường mã sinh viên sid từ hệ thống A và student_id từ hệ thống B thành một trường duy nhất là MaSinhVien trong hệ thống đích.
- **Tích hợp (Integration):** Kết hợp dữ liệu từ nhiều nguồn khác nhau. Ví dụ, nối thông tin học tập của sinh viên với thông tin học phí của họ.
- **Tổng hợp (Aggregation):** Tính toán các giá trị tổng hợp như tổng, trung bình, v.v.

3. **Load (Nạp):** Sau khi đã được biến đổi, dữ liệu sạch và có cấu trúc sẽ được nạp vào hệ thống đích, thường là một Kho dữ liệu (Data Warehouse) hoặc một Data Mart để phục vụ cho các hệ thống OLAP và Business Intelligence.



Hình 2.1.1: Mô hình quy trình ETL điển hình, trích xuất dữ liệu từ các hệ thống nguồn (OLTP) để nạp vào Kho dữ liệu (Data Warehouse) [13]

2.1.3.2 Quy trình ELT (Extract - Load - Transform)

Với sự phát triển của các kho dữ liệu trên nền tảng đám mây (Cloud Data Warehouse) có khả năng xử lý song song cực lớn như Google BigQuery, Amazon Redshift, hay Snowflake, một quy trình mới đã ra đời là ELT.

Trong mô hình ELT, thứ tự các bước được thay đổi: Dữ liệu được trích xuất từ nguồn và được **nạp (Load)** ngay lập tức vào kho dữ liệu ở dạng thô. Sau đó, quá trình **biến**

đổi (Transform) sẽ được thực hiện trực tiếp bên trong kho dữ liệu bằng cách tận dụng năng lực tính toán mạnh mẽ của nó.

Tiêu chí	**ETL (Truyền thống)**	**ELT (Hiện đại)**
Thứ tự	Extract → Transform → Load	Extract → Load → Transform
Nơi biến đổi	Trên một server trung gian (Staging Server)	Trực tiếp trong Kho dữ liệu đích
Thời gian nạp	Chậm hơn, vì phải chờ biến đổi xong	Rất nhanh, nạp dữ liệu thô ngay lập tức
Lưu trữ	Chỉ lưu trữ dữ liệu đã biến đổi	Lưu trữ cả dữ liệu thô và dữ liệu đã biến đổi
Phù hợp với	Dữ liệu có cấu trúc, quy tắc biến đổi rõ ràng	Dữ liệu lớn (Big Data), dữ liệu phi cấu trúc, cần sự linh hoạt

Bảng 2.1.2: So sánh giữa quy trình ETL và ELT

2.1.3.3 Ứng dụng trong bối cảnh trường đại học

Mặc dù quy trình ETL/ELT là cần thiết, trong bối cảnh trường đại học, các phòng ban như Đào tạo, Tài chính, Nhân sự, và Nghiên cứu khoa học thường sử dụng các hệ thống quản lý độc lập, với cấu trúc dữ liệu và quy ước định danh khác nhau. Nếu hệ thống trục tích hợp phải tự mình xử lý từng định dạng riêng biệt, việc mở rộng và bảo trì sẽ trở nên rất phức tạp.

Thay vì vậy, một cách tiếp cận hiệu quả hơn là định nghĩa một mô hình dữ liệu chuẩn (Canonical Data Model) cho toàn trường. Trục tích hợp sẽ công bố các chuẩn dữ liệu, định dạng tệp, hoặc API đầu vào theo mô hình này. Mỗi phòng ban khi gửi dữ liệu lên trục tích hợp phải tuân thủ định dạng chuẩn đó.

Dữ liệu gửi đến trục tích hợp sẽ trải qua các bước:

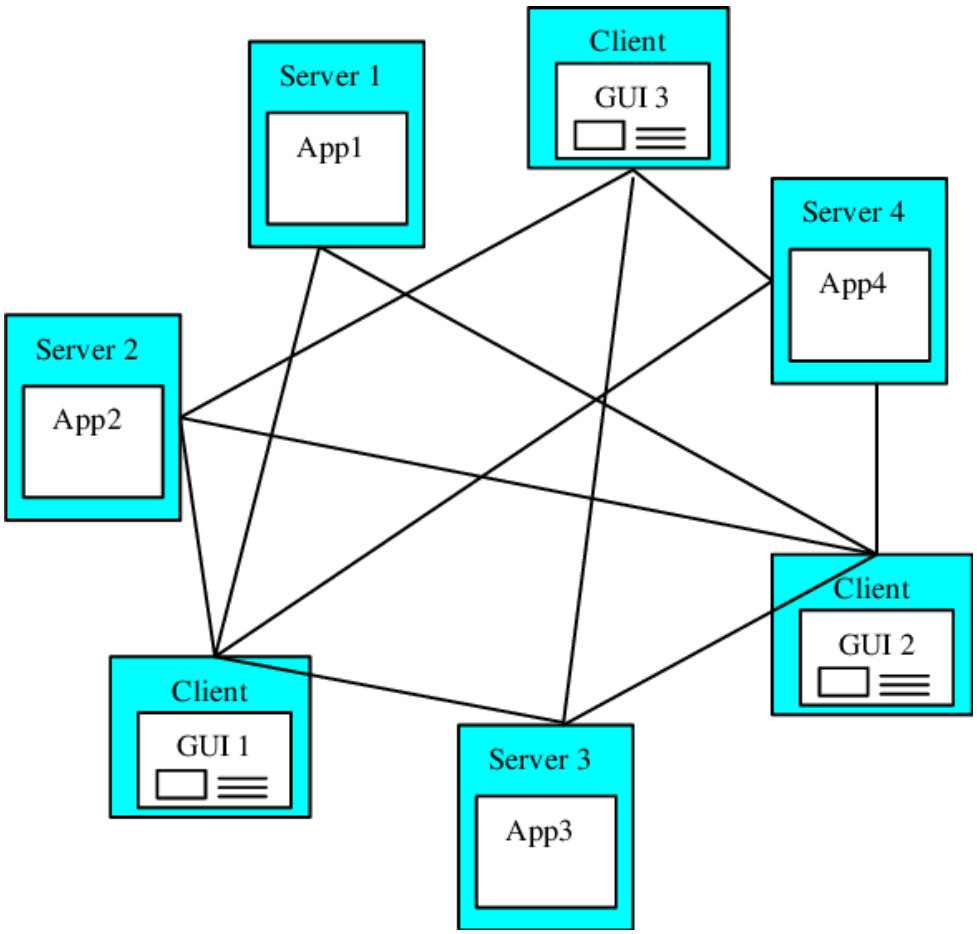
- **Extract:** Tiếp nhận dữ liệu được gửi đến từ các hệ thống nguồn (qua API, file upload, hoặc message queue).
- **Validate:** Kiểm tra cấu trúc, kiểu dữ liệu, và giá trị theo bộ tiêu chuẩn đã được định nghĩa (ví dụ: *MaSV* phải là chuỗi 8 ký tự, ngày sinh phải hợp lệ, mã khoa phải tồn tại trong danh mục).
- **Transform & Load:** Nếu dữ liệu hợp lệ, trục tích hợp sẽ chuyển đổi về mô hình dữ liệu chuẩn nội bộ và nạp vào kho dữ liệu dùng chung.

Với cách tiếp cận này, hệ thống trục tích hợp không cần biết chi tiết về cách từng đơn vị nội bộ tổ chức dữ liệu, mà chỉ cần đảm bảo mọi dữ liệu đầu vào tuân thủ chuẩn thống nhất. Điều này giúp giảm độ phức tạp, đảm bảo tính nhất quán, và tăng khả năng mở rộng của toàn hệ thống.

2.2 Các kiến trúc tích hợp dữ liệu

Việc lựa chọn một kiến trúc tích hợp phù hợp là yếu tố nền tảng quyết định sự thành công, khả năng mở rộng và bảo trì của một hệ thống. Dưới đây là phân tích về các mô hình kiến trúc phổ biến, từ truyền thống đến hiện đại.

2.2.1 Kiến trúc điểm-nối-điểm (Point-to-Point - P2P)



Hình 2.2.1: Kiến trúc Point-to-point [11]

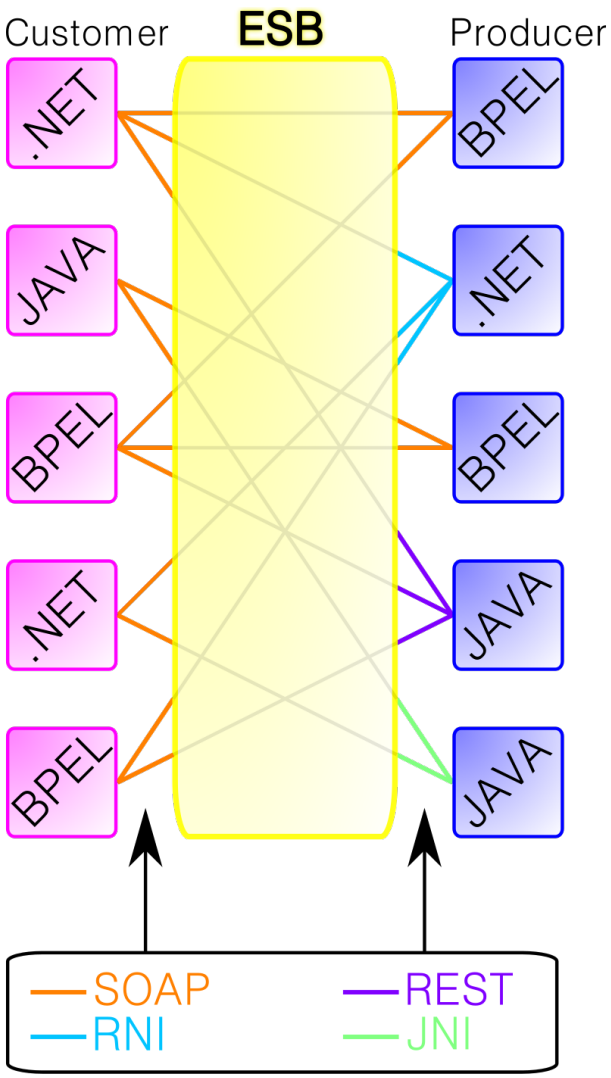
- **Mô tả:** Đây là kiến trúc cơ bản nhất, trong đó mỗi hệ thống được kết nối trực tiếp với một hệ thống khác khi có nhu cầu trao đổi dữ liệu. Nếu hệ thống A cần dữ liệu từ hệ thống B, một kết nối riêng lẻ sẽ được thiết lập giữa chúng.
- **Ưu điểm:** Triển khai nhanh chóng và đơn giản cho các kịch bản có quy mô rất nhỏ (chỉ từ 2 đến 3 hệ thống).
- **Nhược điểm:** Khi số lượng hệ thống tăng lên, số lượng kết nối sẽ tăng theo cấp số nhân, tuân theo công thức $n(n-1)/2$, với n là số hệ thống. Điều này nhanh chóng tạo ra một cấu trúc phức tạp, khó quản lý, khó theo dõi và bảo trì, thường được gọi là "kiến trúc spaghetti"(spaghetti architecture).

2.2.2 Kiến trúc trục và nan hoa (Hub-and-Spoke)

- **Mô tả:** Đây là mô hình kinh điển của một trục tích hợp dữ liệu. Một Hub (trục) trung tâm được thiết lập, và tất cả các hệ thống con (Spokes - nan hoa) đều kết nối vào Hub này thay vì kết nối trực tiếp với nhau. Hub chịu trách nhiệm tiếp nhận, biến đổi, và định tuyến dữ liệu giữa các nan hoa.
- **Ưu điểm:** Cung cấp khả năng quản lý tập trung; giảm thiểu đáng kể số lượng kết nối so với P2P; dễ dàng thêm hoặc bớt các hệ thống con mà không ảnh hưởng đến toàn bộ mạng lưới.
- **Nhược điểm:** Hub trung tâm có nguy cơ trở thành điểm nghẽn cổ chai (single point of failure and bottleneck). Nếu Hub gặp sự cố, toàn bộ luồng dữ liệu trong hệ thống sẽ bị gián đoạn.

2.2.3 Kiến trúc bus dịch vụ doanh nghiệp (Enterprise Service Bus - ESB)

- **Mô tả:** ESB có thể được xem là một sự tiến hóa của mô hình Hub-and-Spoke. Nó hoạt động như một nền tảng trung gian (bus) mà các ứng dụng và dịch vụ có thể kết nối vào. ESB không chỉ đơn thuần truyền tải dữ liệu mà còn cung cấp một loạt các dịch vụ giá trị gia tăng như: điều phối quy trình nghiệp vụ (process orchestration), chuyển đổi giao thức (protocol transformation), định tuyến thông điệp thông minh, và quản lý bảo mật.
- **Ưu điểm:** Rất mạnh mẽ và linh hoạt, có khả năng xử lý các quy trình nghiệp vụ phức tạp, phù hợp cho các doanh nghiệp lớn với hệ sinh thái ứng dụng đa dạng.

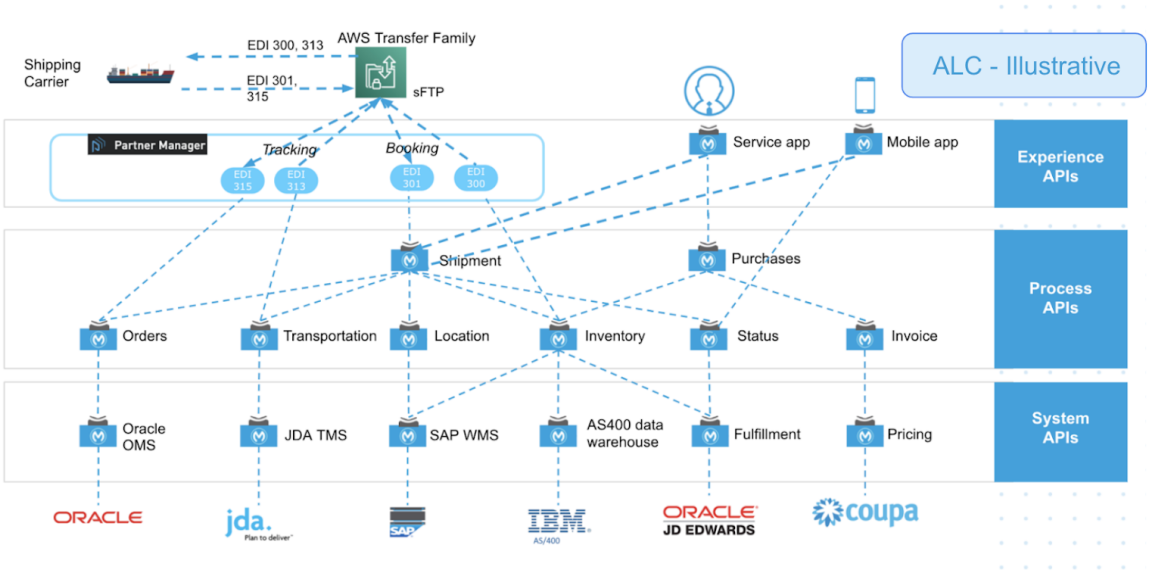


Hình 2.2.2: Kiến trúc Enterprise Service Bus[12]

- **Nhược điểm:** Việc triển khai và vận hành một hệ thống ESB thường rất phức tạp và tốn kém, đòi hỏi chuyên môn kỹ thuật cao.

2.2.4 Kiến trúc kết nối dựa trên API (API-led Connectivity)

- **Mô tả:** Đây là một phương pháp tiếp cận hiện đại, tập trung vào việc thiết kế và xây dựng các Giao diện lập trình ứng dụng (API) có khả năng tái sử dụng cao. Thay vì xây dựng một hệ thống trung tâm nguyên khối, kiến trúc này phân lớp các API thành ba tầng riêng biệt[10]:
  - **System APIs (API Hệ thống):** Lớp dưới cùng, có nhiệm vụ trừu tượng hóa (abstract) và mở khóa dữ liệu từ các hệ thống lõi (ví dụ: CSDL sinh viên, hệ



Hình 2.2.3: Minh hoạ cho kiến trúc API-led Connectivity [10]

thống tài chính). Các API này chỉ tập trung vào việc cung cấp dữ liệu thô một cách an toàn và nhất quán.

- **Process APIs (API Quy trình):** Lớp giữa, thực hiện việc điều phối và kết hợp dữ liệu từ nhiều System API để tạo ra các chức năng nghiệp vụ cụ thể. Ví dụ, một API "Tổng hợp thông tin sinh viên"sẽ gọi đến System API từ hệ thống đào tạo, tài chính và ký túc xá để trả về một bản ghi hoàn chỉnh.
- **Experience APIs (API Trải nghiệm):** Lớp trên cùng, được thiết kế để phục vụ cho một kênh hoặc một trải nghiệm người dùng cuối cụ thể, ví dụ: API cho ứng dụng di động của sinh viên, API cho cổng thông tin dành cho giảng viên.

- **Ưu điểm:** Thúc đẩy tính linh hoạt, khả năng tái sử dụng và tốc độ phát triển; phù hợp với các kiến trúc hiện đại như Microservices; cho phép phát triển và mở rộng một cách phân tán.

2.2.5 So sánh và lựa chọn kiến trúc cho bối cảnh đại học

Sau khi phân tích các mô hình, việc lựa chọn kiến trúc API-led Connectivity được xem là phù hợp nhất cho bối cảnh của một trường đại học hiện đại. Dưới đây là các lập luận chi tiết.

2.2.5.1 Những hạn chế của kiến trúc truyền thống

Các mô hình như Hub-and-Spoke và ESB, mặc dù có giá trị trong nhiều ngữ cảnh, lại bộc lộ những nhược điểm đáng kể khi áp dụng vào một môi trường đòi hỏi sự linh hoạt và đổi mới nhanh như trường đại học:

- **Tạo ra kiến trúc trung tâm nguyên khối (Monolithic Hub):** Toàn bộ logic tích hợp (biến đổi, định tuyến) bị tập trung vào một Hub hoặc Bus duy nhất. Điều này làm cho thành phần trung tâm trở nên phức tạp, khó phát triển, bảo trì và nâng cấp.
- **Điểm nghẽn và rủi ro hệ thống (bottleneck):** Vì mọi luồng dữ liệu đều phải đi qua điểm trung tâm, hệ thống rất dễ bị quá tải khi lưu lượng tăng cao. Sự cố tại trung tâm sẽ làm tê liệt toàn bộ hệ thống.
- **Hạn chế tính linh hoạt và khả năng đáp ứng nhanh (Lack of Agility):** Khi có nhu cầu phát triển một ứng dụng mới (ví dụ: ứng dụng cho tuần lễ hướng nghiệp), các nhóm phát triển phải phụ thuộc vào đội ngũ quản lý hệ thống trung tâm để xây dựng các kết nối và logic mới. Quá trình này thường chậm chạp và cản trở sự đổi mới.
- **Sự phức tạp và chi phí của ESB:** Một hệ thống ESB đầy đủ chức năng thường quá cồng kềnh và tốn kém, vượt quá nhu cầu và quy mô của hầu hết các trường đại học, đồng thời có nguy cơ phụ thuộc vào nhà cung cấp (vendor lock-in).

2.2.5.2 Lý do lựa chọn kiến trúc API-led Connectivity

Kiến trúc kết nối dựa trên API giải quyết trực tiếp các vấn đề nêu trên và mang lại nhiều lợi ích chiến lược:

- **Thúc đẩy tái sử dụng và tăng tốc độ phát triển:** Thay vì xây dựng các kết nối dùng một lần, kiến trúc này tạo ra một danh mục các API có thể được tái sử dụng trên nhiều dự án. Ví dụ, System API "Thông tin sinh viên"sau khi được xây dựng có thể phục vụ cho cả ứng dụng di động, cổng thông tin giảng viên, và hệ thống thư viện mà không cần phát triển lại, giúp giảm thời gian triển khai các sáng kiến mới một cách đáng kể.
- **Phân tán và dễ dàng mở rộng (Decentralized & Scalable):** Logic tích hợp không còn bị dồn vào một điểm. Mỗi API là một thành phần độc lập, có thể được

phát triển, triển khai và mở rộng riêng lẻ bởi các nhóm khác nhau. Điều này hoàn toàn phù hợp với triết lý của kiến trúc Microservices.

- **Tăng cường bảo mật và quản trị:** Mỗi API hoạt động như một cổng kiểm soát (gateway). Thông qua một API Gateway, nhà trường có thể dễ dàng áp dụng các chính sách bảo mật, giới hạn tần suất truy cập, và kiểm soát quyền truy cập dữ liệu cho từng ứng dụng một cách chi tiết và an toàn.

- **Sẵn sàng cho hệ sinh thái mở và tương lai:** Kiến trúc này giúp trường đại học không chỉ tích hợp hiệu quả các hệ thống nội bộ mà còn dễ dàng kết nối với các hệ thống bên ngoài (như hệ thống của Bộ Giáo dục và Đào tạo), các đối tác doanh nghiệp, hoặc tích hợp các công nghệ mới (IoT, AI/ML) bằng cách tạo ra các API mới mà không làm xáo trộn kiến trúc hiện có.

Chương 3

Thiết kế kiến trúc

Dựa trên các nghiên cứu về lý thuyết và các mô hình kiến trúc đã được trình bày ở chương trước, chương này sẽ đi sâu vào việc thiết kế kiến trúc chi tiết cho hệ thống Trục tích hợp dữ liệu của trường đại học, sử dụng kiến trúc API-led Connectivity.

3.1 Yêu cầu chức năng và phi chức năng

3.1.1 Yêu cầu chức năng (Functional Requirements)

Hệ thống cần phải đáp ứng các yêu cầu chức năng cốt lõi sau:

- **Tích hợp dữ liệu từ nhiều nguồn:** Hệ thống phải có khả năng kết nối và trích xuất dữ liệu từ các hệ thống nguồn đa dạng trong trường (hệ thống quản lý đào tạo, tài chính, nhân sự, thư viện, ký túc xá, v.v.).
- **Cung cấp dữ liệu hợp nhất qua API:** Hệ thống phải cung cấp một bộ các API an toàn, được tài liệu hóa rõ ràng để các ứng dụng bên ngoài (Cổng thông tin sinh viên MyBK, hệ thống LMS, báo cáo HEMIS) có thể truy vấn và sử dụng dữ liệu đã được hợp nhất.
- **Lập lịch đồng bộ tự động:** Hệ thống phải cho phép cấu hình và thực thi các tác vụ đồng bộ dữ liệu một cách tự động theo lịch trình định sẵn (ví dụ: hàng đêm, hàng tuần).
- **Quản lý và giám sát tập trung:** Cung cấp một giao diện quản trị (Dashboard) cho phép người quản trị theo dõi trạng thái hoạt động của các luồng dữ liệu, quản lý người dùng, phân quyền và cấu hình các tác vụ tích hợp.

- **Xác thực và phân quyền truy cập:** Mọi yêu cầu truy cập dữ liệu thông qua API đều phải được xác thực và tuân thủ các quy tắc phân quyền đã được định nghĩa để đảm bảo chỉ những người dùng/ứng dụng hợp lệ mới có thể truy cập dữ liệu tương ứng.

3.1.2 Yêu cầu phi chức năng (Non-functional Requirements)

Bên cạnh các chức năng, hệ thống cần đảm bảo các đặc tính quan trọng sau:

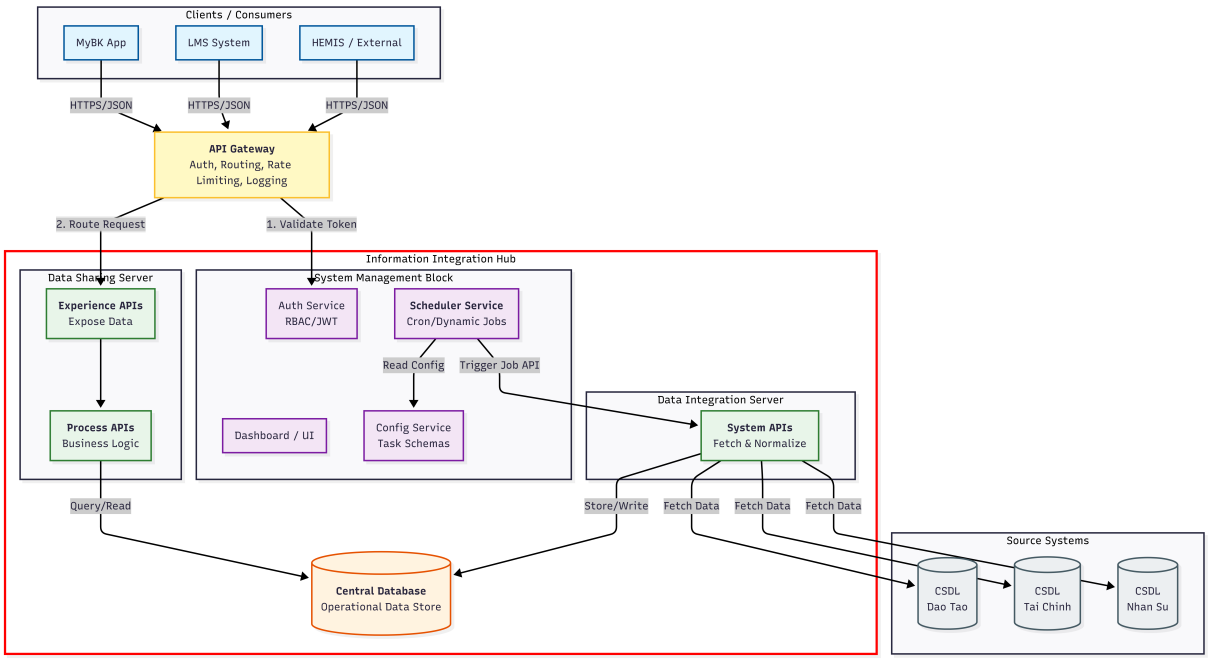
- **Tính bảo mật (Security):** Dữ liệu, đặc biệt là thông tin cá nhân của sinh viên và giảng viên, phải được bảo vệ nghiêm ngặt ở cả trạng thái tĩnh (at-rest) và trạng thái động (in-transit). Hệ thống phải có cơ chế chống lại các hình thức tấn công phổ biến.
- **Tính mở rộng (Scalability):** Kiến trúc phải cho phép mở rộng dễ dàng khi có thêm các hệ thống nguồn mới hoặc khi lưu lượng truy cập từ các ứng dụng người dùng cuối tăng cao. Các thành phần phải có khả năng mở rộng độc lập.
- **Tính tái sử dụng (Reusability):** Các thành phần tích hợp, đặc biệt là các API, phải được thiết kế để có thể tái sử dụng cho nhiều mục đích và ứng dụng khác nhau, nhằm giảm thiểu thời gian và chi phí phát triển trong tương lai.
- **Tính sẵn sàng (Availability):** Hệ thống phải hoạt động ổn định và có độ sẵn sàng cao để không làm gián đoạn hoạt động của các ứng dụng phụ thuộc vào nó.
- **Khả năng bảo trì (Maintainability):** Kiến trúc phải được phân tách rõ ràng, các thành phần có trách nhiệm duy nhất, giúp cho việc bảo trì, sửa lỗi và nâng cấp trở nên dễ dàng hơn.

3.2 Sơ đồ kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống được đề xuất như trong Hình 3.2.1.

3.3 Giải thích chi tiết các thành phần

Kiến trúc được chia thành các khối thành phần chính, mỗi khối có một vai trò và trách nhiệm riêng biệt, tuân thủ theo mô hình API-led Connectivity ba lớp.



Hình 3.2.1: Sơ đồ kiến trúc tổng thể của Trục tích hợp dữ liệu

3.3.1 Các hệ thống nguồn (Data Sources)

Đây là tập hợp các cơ sở dữ liệu và hệ thống nghiệp vụ hiện có trong trường, chứa đựng các dữ liệu gốc cần được tích hợp. Mỗi hệ thống này sẽ cung cấp dữ liệu thông qua một lớp API riêng để đảm bảo tính an toàn và sự tách biệt, thay vì cho phép kết nối trực tiếp vào cơ sở dữ liệu.

3.3.2 Trục tích hợp thông tin (Information Integration Hub)

Đây là trái tim của toàn bộ hệ thống, được bao bọc bởi đường viền màu đỏ trong sơ đồ. Nó bao gồm các thành phần con sau:

3.3.2.1 Máy chủ Tích hợp Dữ liệu (Data Integration Server)

Thành phần này chịu trách nhiệm cho luồng dữ liệu đầu vào (ingestion flow), hiện thực hóa lớp **System API**:

- **System API:** Đây là tập hợp các API nội bộ, có nhiệm vụ kết nối đến các API của hệ thống nguồn để trích xuất dữ liệu thô. Vai trò của nó là thu thập và trừu tượng hóa các hệ thống lõi, tạo ra một cách thức truy cập dữ liệu nhất quán.
- Sau khi trích xuất, máy chủ này sẽ thực hiện các bước biến đổi dữ liệu cơ bản

(ETL/ELT) và nạp (stores) dữ liệu đã được làm sạch, chuẩn hóa vào Kho dữ liệu trung tâm.

3.3.2.2 Máy chủ Chia sẻ Dữ liệu (Data Sharing Server)

Thành phần này chịu trách nhiệm cho luồng dữ liệu đầu ra (query flow), hiện thực hóa hai lớp **Process API** và **Experience API**:

- **Process API:** Lớp này chứa các logic nghiệp vụ phức tạp. Nó thực hiện truy vấn (queries) đến Kho dữ liệu trung tâm, kết hợp và tổng hợp thông tin để tạo ra các thực thể dữ liệu có giá trị nghiệp vụ cao.
- **Experience API:** Lớp trên cùng, có nhiệm vụ trả về các dữ liệu đã được xử lý bởi Process API ra cho các ứng dụng người dùng cuối. Các API ở lớp này được thiết kế tối ưu cho từng trải nghiệm cụ thể (ví dụ: API cho ứng dụng di động sẽ chỉ trả về những thông tin cần thiết nhất để tối ưu tốc độ).

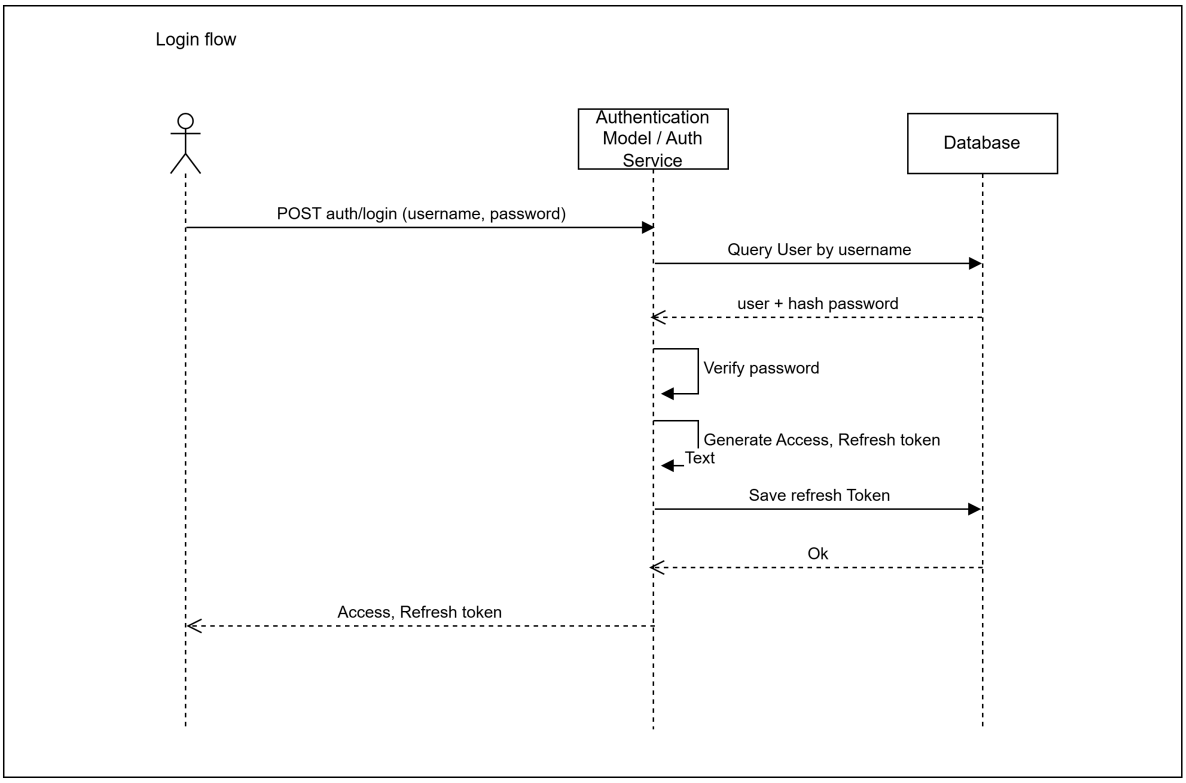
3.3.2.3 Kho dữ liệu trung tâm (Central Data Store)

Đây là nơi lưu trữ dữ liệu đã được tích hợp và hợp nhất từ tất cả các nguồn. Nó đóng vai trò là cơ sở dữ liệu duy nhất cho toàn bộ các ứng dụng trong trường.

3.3.3 Cổng API (API Gateway)

API Gateway là cổng vào duy nhất cho tất cả các yêu cầu từ bên ngoài (MyBK, LMS, HEMIS) đến hệ thống. Nó chịu trách nhiệm:

- **Xác thực (Authenticating):** Kiểm tra danh tính của người dùng/ứng dụng gọi đến.
- **Định tuyến (Routing):** Chuyển tiếp yêu cầu hợp lệ đến Experience API tương ứng trên Máy chủ Chia sẻ Dữ liệu.
- **Kiểm soát (Validating):** Áp dụng các chính sách bảo mật, giới hạn tần suất truy cập (rate limiting) để bảo vệ hệ thống.



Hình 3.3.1: Login Flow cho hệ thống

3.3.4 Thiết kế chi tiết Module Quản lý Hệ thống

Module Quản lý Hệ thống (System Management) là thành phần cốt lõi đảm bảo an ninh, quản trị và khả năng kiểm soát cho toàn bộ Trục tích hợp dữ liệu. Thay vì chỉ là một giao diện Dashboard đơn thuần, nó bao gồm một dịch vụ Xác thực và Phân quyền (Authentication and Authorization - Auth Service) mạnh mẽ, được thiết kế theo mô hình minh họa trong Hình **??**.

3.3.4.1 Luồng Xác thực (Flow)

Xác thực là quá trình xác minh danh tính của một người dùng hoặc một hệ thống. Luồng xử lý được thực hiện như sau:

1. **Yêu cầu đăng nhập:** Người dùng (User) gửi thông tin định danh (ví dụ: username và password) đến ứng dụng backend (NestJS).

2. **Xác minh thông tin:** Ứng dụng backend truy vấn vào Cơ sở dữ liệu (DataBase), cụ thể là bảng `Users`, để kiểm tra tính hợp lệ của thông tin đăng nhập.

3. **Tạo và trả về JWT:** Nếu thông tin hợp lệ, hệ thống sẽ tạo ra một chuỗi JSON Web Token (JWT). JWT này là một chuỗi được mã hóa, chứa các thông tin (claims) về người dùng như ID, vai trò (role), và thời gian hết hạn. Token này sau đó được gửi về cho client (người dùng).

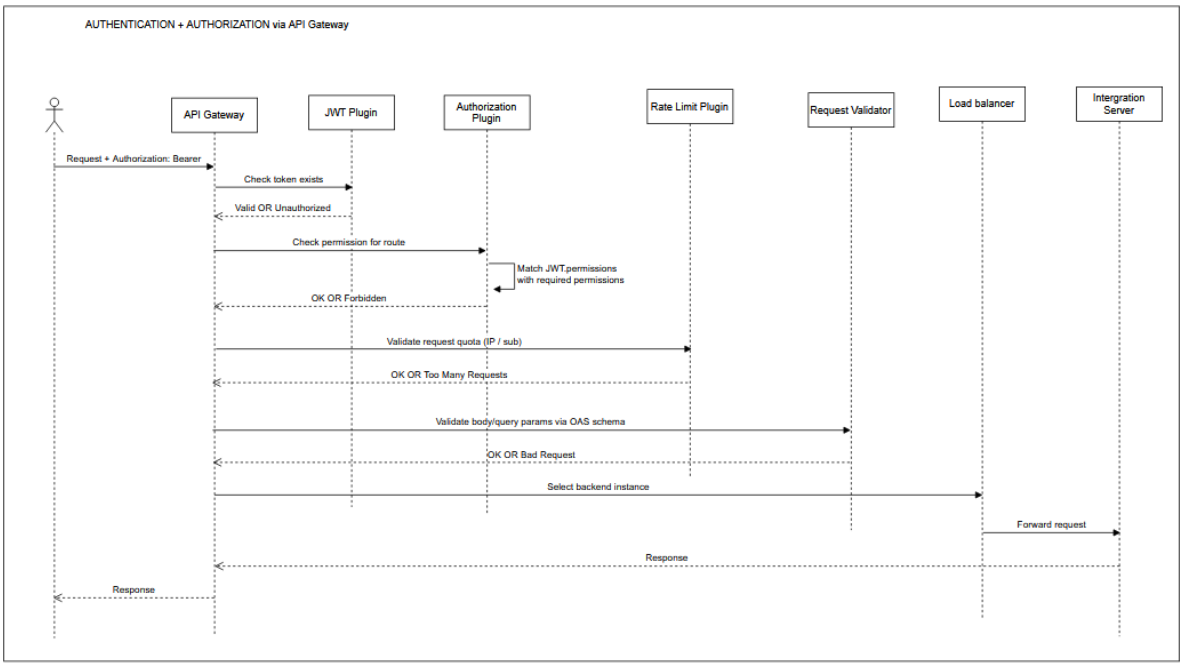
Việc sử dụng JWT giúp cho hệ thống trở nên "stateless"(phi trạng thái), vì mọi thông tin cần thiết cho các phiên làm việc sau này đều được lưu trữ an toàn ngay trên client, giảm tải cho máy chủ.

3.3.4.2 Luồng Phân quyền (Authorization Flow)

Phân quyền là quá trình quyết định xem một người dùng đã được xác thực có quyền thực hiện một hành động cụ thể hay không. Quá trình này diễn ra ở mọi yêu cầu truy cập tài nguyên sau khi người dùng đã đăng nhập thành công:

1. **Gửi yêu cầu kèm JWT:** Client gửi mỗi yêu cầu lên máy chủ, đính kèm JWT trong phần Header (thường là ‘Authorization: Bearer <token>‘).

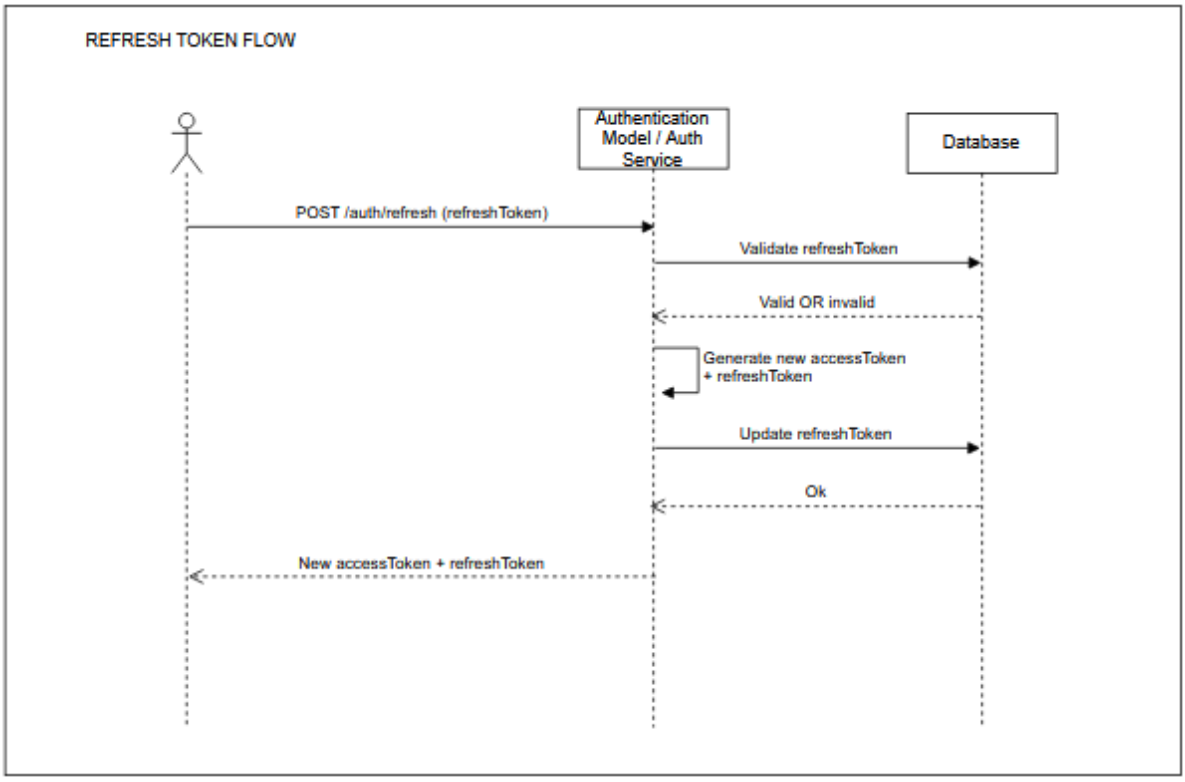
2. **Xác thực Token:** API Gateway hoặc một lớp middleware trong ứng dụng NestJS sẽ chặn yêu cầu lại, giải mã và xác thực chữ ký của JWT để đảm bảo token là hợp lệ và chưa bị sửa đổi.



Hình 3.3.2: Luồng xử lý Xác thực của hệ thống

3. **Kiểm tra quyền hạn (RBAC):** Sau khi xác thực token thành công, hệ thống sẽ thực hiện Phân quyền dựa trên mô hình **Role-Based Access Control (RBAC)**.

   - Hệ thống đọc thông tin **Vai trò (Role)** của người dùng được đính kèm trong JWT (ví dụ: "Giảng viên", "Sinh viên", "Quản trị viên").
   - Dựa trên Vai trò đó, hệ thống truy vấn vào cơ sở dữ liệu để xác định danh sách các **Quyền (Permissions)** mà vai trò đó sở hữu. Ví dụ, vai trò "Giảng viên"có quyền "Xem danh sách lớp học của mình", "Nhập điểm", nhưng không có quyền "Sửa học phí".
   - Hệ thống so sánh quyền yêu cầu của hành động với danh sách quyền mà người dùng có. Nếu phù hợp, yêu cầu được chấp nhận; nếu không, yêu cầu bị từ chối.



Hình 3.3.3: Luồng xử lý refresh token của hệ thống

3.3.4.3 Cấu trúc Cơ sở dữ liệu cho RBAC

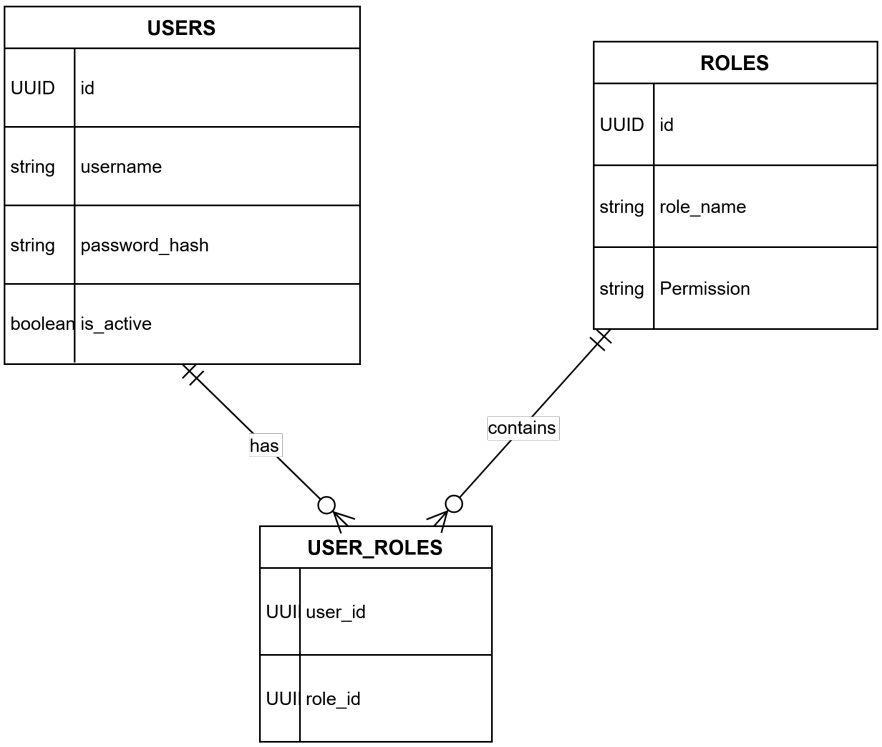
Để hiện thực hóa mô hình RBAC, cơ sở dữ liệu được thiết kế với các bảng chính sau:

- **Users:** Lưu trữ thông tin định danh của người dùng (username, password đã được hash, thông tin cá nhân...).
- **Roles:** Định nghĩa các vai trò trong hệ thống.

Mối quan hệ nhiều-nhiều giữa chúng được thể hiện qua các bảng trung gian (không có trong sơ đồ nhưng là một phần của thiết kế):

- **user_roles:** xạ người dùng nào có vai trò nào.

Cấu trúc này mang lại sự linh hoạt tối đa, cho phép người quản trị có thể dễ dàng tạo ra các vai trò mới hoặc thay đổi quyền hạn của một vai trò mà không cần can thiệp vào mã nguồn của ứng dụng, thông qua giao diện Dashboard của Module Quản lý Hệ thống.



Hình 3.3.4: RBAC

Chương 4

Các công nghệ sử dụng trong hệ thống

4.1 Nền tảng Backend: NestJS

NestJS là framework backend hiện đại xây dựng trên Node.js và được sử dụng để triển khai Data Integration Server và Data Sharing Server của trục tích hợp dữ liệu. Lựa chọn NestJS mang lại các lợi ích chính:

- **Hiệu năng cao và dễ mở rộng** NestJS dựa trên mô hình bất đồng bộ của Node.js và có thể chạy với Express hoặc Fastify, đảm bảo xử lý lượng lớn yêu cầu từ MyBK App, LMS và các dịch vụ nội bộ. Hệ thống cũng dễ dàng mở rộng theo chiều ngang bằng Docker hoặc Kubernetes.
- **Kiến trúc Module hóa – dễ phát triển và bảo trì**. NestJS áp dụng kiến trúc module theo domain (Student, Course, LMS, HR, ...), giúp mã nguồn rõ ràng, dễ

mở rộng và phù hợp cho các dự án lớn. TypeScript hỗ trợ kiểm tra kiểu dữ liệu tĩnh, giảm lỗi runtime.

- **Xác thực và chuẩn hóa dữ liệu mạnh mẽ**. NestJS hỗ trợ các thư viện class-validator, class-transformer, Zod/Joi để định nghĩa và kiểm tra schema dữ liệu. Điều này đặc biệt quan trọng khi ingest dữ liệu từ SIS, LMS, HRM và các hệ thống nguồn khác vào trục tích hợp.
- **Dependency Injection mạnh – theo chuẩn enterprise** Cơ chế DI của NestJS giúp tách biệt rõ logic nghiệp vụ, data access và adapter tích hợp, góp phần tăng độ linh hoạt và khả năng kiểm thử.
- **Tự động sinh tài liệu API – phù hợp kiến trúc API-led**. NestJS tích hợp Swagger/OpenAPI để tự động sinh tài liệu API, đảm bảo API Contract luôn rõ ràng và đồng bộ – một yếu tố quan trọng trong kiến trúc API-led Connectivity.

Tiêu chí	**NestJS**	**FastAPI**	**Spring Boot**
Ngôn ngữ	TypeScript (Node.js)	Python	Java
Hiệu năng	Cao, bất đồng bộ	Cao, dùng Starlette/Uvicorn	Rất cao nhưng nặng
Kiến trúc	Module hoá, giống Spring	Không có module pattern chuẩn	Enterprise mạnh mẽ
Tốc độ phát triển	Nhanh, TS hỗ trợ tốt	Nhanh nhất	Chậm hơn do Java verbose
ORM hỗ trợ	Prisma, TypeORM	SQLAlchemy	JPA/Hibernate
Tài liệu API	Swagger tự động	Swagger tự động	Swagger phụ thuộc config
Phù hợp trục tích hợp	X		

Bảng 4.1.1: So sánh các backend framework

4.2 ORM & Database Migration Tool cho NestJS: Prisma

- **Prisma ORM** Vai trò: Công cụ ORM giúp NestJS tương tác với các cơ sở dữ liệu quan hệ như PostgreSQL, MySQL, SQL Server hoặc MongoDB.

  Lý do chọn Prisma: Hiệu năng cao, cú pháp truy vấn đơn giản. Hỗ trợ TypeScript mạnh mẽ (type-safe). Tự động sinh type → giảm lỗi runtime. Phù hợp với mô hình chuẩn hóa dữ liệu trong trục tích hợp.

- **Prisma Migrate (Migration Tool)** Vai trò: Quản lý thay đổi schema của Data Hub thông qua các file migration tự động, giúp: Theo dõi phiên bản schema; Đồng bộ schema giữa các môi trường ; Thực hiện migrate/rollback an toàn

  Ưu điểm: Tích hợp chặt với Prisma ORM, tự động sinh SQL migration từ file schema, dễ dùng trong CI/CD.

Tiêu chí	Prisma	TypeORM	Sequelize
Kiểu ORM	Schema-first	Decorator-based	Active Record
Type-safety	Rất tốt (tự sinh type)	Tốt	Trung bình
Xu hướng phát triển	Hiện đại, mạnh	Ổn định	Cũ, ít phát triển
Cú pháp truy vấn	Rõ ràng, dễ đọc	Khá phức tạp	Ngắn gọn nhưng ít an toàn
Migration	Prisma Migrate tự động	Không ổn định, dễ lỗi	Kém linh hoạt
Hiệu năng	Cao	Khá cao	Trung bình
Phù hợp Data Hub	X		

Bảng 4.2.1: So sánh các ORM phổ biến trong hệ sinh thái Nest.js

4.3 Cơ sở dữ liệu trung tâm: PostgreSQL

Lý do chọn PostgreSQL. Tính toàn vẹn và nhất quán dữ liệu: Dữ liệu của một trường đại học có tính cấu trúc và quan hệ rất cao (sinh viên thuộc về lớp, lớp thuộc về khoa, sinh viên đăng ký môn học...). PostgreSQL, với cơ chế ràng buộc khóa ngoại

(foreign keys), transactions (ACID), đảm bảo tính toàn vẹn và nhất quán của dữ liệu ở mức cao nhất, điều mà các CSDL NoSQL thường đánh đổi để lấy sự linh hoạt.

Truy vấn phức tạp. Nhu cầu báo cáo, thống kê trong môi trường đại học thường đòi hỏi các truy vấn phức tạp, kết hợp dữ liệu từ nhiều bảng. Ngôn ngữ SQL của PostgreSQL cực kỳ mạnh mẽ và được tối ưu cho các tác vụ này.

Tính linh hoạt với JSONB: PostgreSQL hỗ trợ kiểu dữ liệu JSONB, cho phép lưu trữ và truy vấn dữ liệu dạng JSON một cách hiệu quả. Điều này mang lại sự linh hoạt cần thiết để lưu trữ các thuộc tính dữ liệu không đồng nhất từ các hệ thống nguồn, kết hợp ưu điểm của cả hai thế giới SQL và NoSQL.

Tiêu chí	**PostgreSQL**	**MySQL**	**MongoDB**
Loại CSDL	Quan hệ (RDBMS)	Quan hệ (RDBMS)	NoSQL (Document)
Mức độ ACID	Rất mạnh	Trung bình	Không phải ACID hoàn toàn
Hỗ trợ JSON	JSONB mạnh mẽ	JSON yếu hơn	Native JSON
Khả năng mở rộng	Tốt	Tốt	Rất tốt
Truy vấn phức tạp	Rất tốt (window function, CTE)	Tốt	Yếu
Phù hợp Data Hub	X		

Bảng 4.3.1: So sánh các hệ quản trị cơ sở dữ liệu

4.4 Giao diện Quản trị (System Management Dashboard): Next.js & TailwindCSS

- **Nextjs:** một framework của thư viện frontend React phổ biến, có các tính nănnăng Phát triển giao diện phức tạp: System Management Dashboard sẽ bao gồm nhiều biểu đồ, bảng biểu, form cấu hình. Next.js, với hệ sinh thái React phong phú, cung cấp nền tảng vững chắc để xây dựng các thành phần UI phức tạp và có tính tương tác cao.

- **Tối ưu hiệu năng:** Các tính năng tích hợp sẵn như Server-Side Rendering (SSR) và Static Site Generation (SSG) giúp trang dashboard có tốc độ tải ban đầu rất nhanh, mang lại trải nghiệm tốt cho người quản trị.
- **Cấu trúc dự án rõ ràng:** Tính năng File-based routing giúp tổ chức các trang quản lý một cách trực quan và dễ dàng mở rộng.

  TailwindCSS: Là một framework utility-first CSS, giúp tăng tốc quá trình xây dựng giao diện tùy chỉnh một cách nhất quán và hiệu quả, cho phép tập trung vào chức năng thay vì viết CSS thủ công.

4.5 Kong API Gateway

- **Quản lý tập trung:** Kong cung cấp một điểm quản lý duy nhất cho tất cả các API, giúp dễ dàng theo dõi và kiểm soát luồng dữ liệu vào/ra hệ thống.

- **Bảo mật và Xác thực:** Dễ dàng tích hợp các cơ chế bảo mật thông qua các plugin như JWT, OAuth 2.0, API Keys... để đảm bảo chỉ những người dùng và ứng dụng hợp lệ mới có quyền truy cập vào dữ liệu.
- **Hiệu năng cao và Khả năng mở rộng:** Kong được xây dựng cho hiệu năng cao, độ trễ thấp và có thể mở rộng theo chiều ngang để đáp ứng lưu lượng truy cập lớn.
- **Giám sát và Điều tiết:** Cung cấp các tính năng quan trọng như Rate Limiting (giới hạn tần suất truy cập để chống tấn công DoS) và Logging (ghi lại nhật ký truy cập), tích hợp liền mạch với các công cụ giám sát để theo dõi "sức khỏe"của hệ thống.

Tiêu chí	**Kong**	**NGINX**	**Traefik**
Mục đích chính	API Gateway	Reverse Proxy	Reverse Proxy / Gateway
Plugin	Rất nhiều (JWT, Rate limit, Keycloak...)	Ít	Trung bình
Hiệu năng	Rất cao	Rất cao	Cao
Khả năng mở rộng	Rất tốt	Tốt	Rất tốt
Tính năng API Management	Đầy đủ	Không có	Cơ bản
Phù hợp trục tích hợp	X		

Bảng 4.5.1: So sánh các API Gateway phổ biến

Chương 5

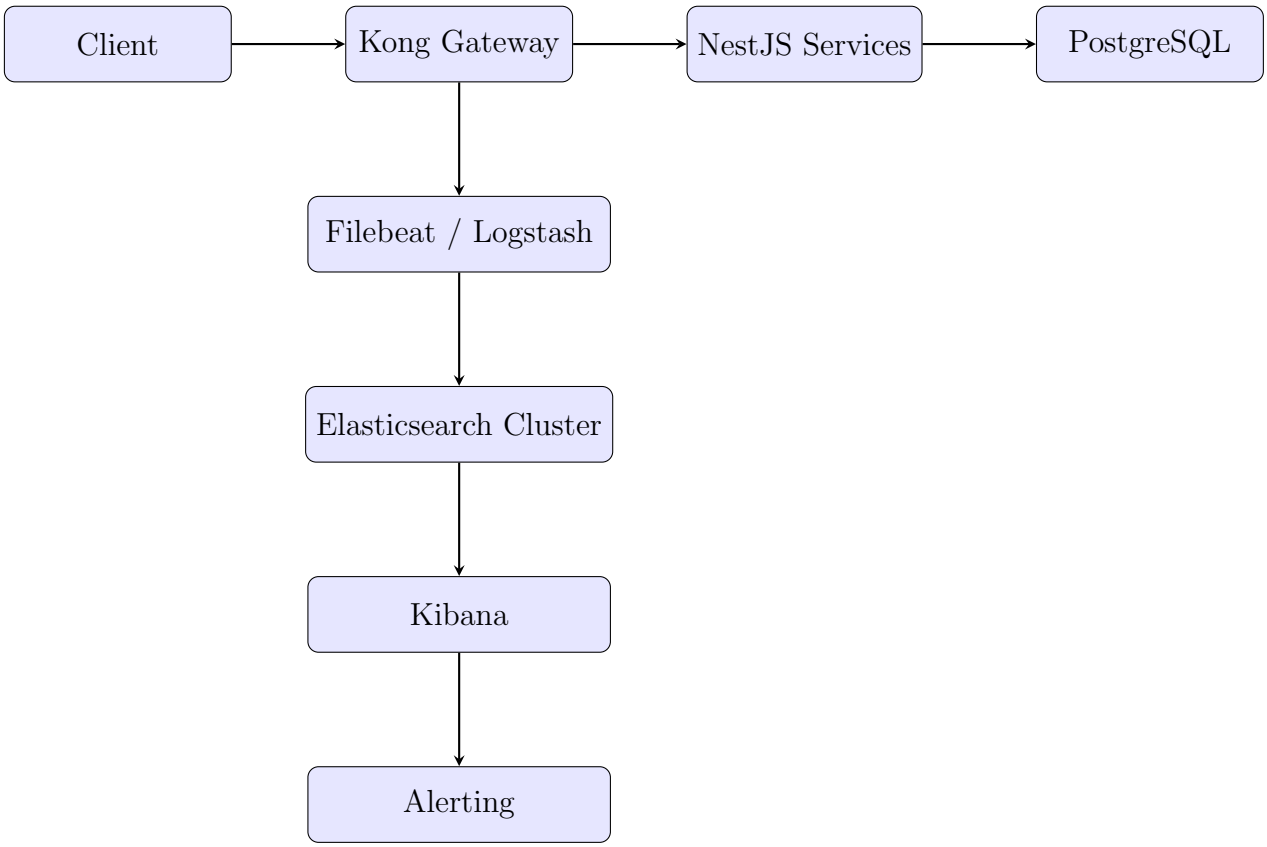
Giám sát, Ghi log và Cảnh báo hệ thống

Trong kiến trúc trục tích hợp dữ liệu phân tán, khả năng quan sát toàn diện (Observability) đóng vai trò then chốt. Nó không chỉ giúp theo dõi hiệu năng (Performance Monitoring) mà còn là công cụ chính để truy vết lỗi (Troubleshooting), giám sát bảo mật và phân tích hành vi hệ thống. Để đáp ứng các yêu cầu này, nhóm thực hiện triển khai giải pháp giám sát tập trung dựa trên **Elastic Stack (ELK)**, kết hợp giữa thu thập Log và Metric.

5.1 Kiến trúc Observability tổng quan

Hệ thống giám sát được thiết kế theo mô hình tập trung hóa, bao gồm các thành phần chính:

- **Data Sources (Nguồn dữ liệu):** Bao gồm Kong Gateway, Data Integration Server (NestJS), Database và Hệ điều hành máy chủ.
- **Data Shippers (Bộ thu thập):** Sử dụng **Filebeat** để thu thập log và **Metricbeat** để thu thập chỉ số tài nguyên hệ thống.
- **Indexing & Storage: Elasticsearch** đóng vai trò là kho lưu trữ dữ liệu log phân tán và hỗ trợ truy vấn full-text search.
- **Visualization: Kibana** cung cấp giao diện dashboard để trực quan hóa dữ liệu và quản lý cảnh báo.



Hình 5.1.1: Kiến trúc dòng chảy dữ liệu giám sát (Log Flow)

5.2 Chiến lược ghi log (Logging Strategy)

Một thách thức lớn trong hệ thống microservices là khả năng truy vết (trace) một yêu cầu đi qua nhiều thành phần. Hệ thống áp dụng các chiến lược sau:

5.2.1 Structured Logging (Log có cấu trúc)

Thay vì ghi log dạng văn bản thuần túy (plain text), toàn bộ log ứng dụng được chuẩn hóa dưới dạng **JSON**. Định dạng này giúp Elasticsearch đánh chỉ mục (index) dễ dàng và cho phép lọc dữ liệu chính xác. Một mẫu log chuẩn bao gồm:

- `@timestamp`: Thời gian xảy ra sự kiện.
- `level`: Mức độ (INFO, WARN, ERROR).
- `correlation_id`: Mã định danh duy nhất của request (quan trọng).
- `service`: Tên dịch vụ (vd: data-integration-service).

- `context`: Ngữ cảnh (vd: StudentModule, AuthGuard).
- `message`: Nội dung log.

5.2.2 Distributed Tracing với Correlation ID

Mỗi request khi đi vào từ **Kong Gateway** sẽ được gán một `X-Request-ID`. ID này được truyền xuống các service phía sau (NestJS) thông qua HTTP Header. Nhờ đó, quản trị viên có thể tìm kiếm theo ID này trên Kibana để xem toàn bộ hành trình của request từ khi vào cổng đến khi lưu xuống database.

5.3 Các thành phần thu thập chi tiết

5.3.1 Kong API Gateway Logs

Kong được cấu hình sử dụng plugin *File-Log* hoặc *TCP-Log* để xuất log truy cập. Thông tin thu thập bao gồm: Latency (độ trễ), Upstream service, Status code và Consumer ID (người gọi API).

5.3.2 Application Logs (NestJS)

Sử dụng thư viện **Winston** để quản lý log. Winston được cấu hình để tự động che (mask) các dữ liệu nhạy cảm (PII) như mật khẩu, email sinh viên trước khi ghi xuống file. Log bao gồm log nghiệp vụ (Business logic) và log lỗi (Exceptions).

5.3.3 Database & System Logs

- **PostgreSQL:** Kích hoạt module `log_min_duration_statement` để ghi lại các truy vấn chậm (Slow queries) vượt quá 200ms, giúp tối ưu hóa hiệu năng.
- **Metricbeat:** Thu thập các chỉ số hạ tầng như CPU Usage, Memory, Disk I/O và Network Traffic từ Docker Container để phát hiện điểm nghẽn hệ thống.

5.4 Dashboard giám sát và Cảnh báo

5.4.1 Dashboard (Kibana)

Hệ thống trực quan hóa dữ liệu thông qua các Dashboard thời gian thực:

1. **API Performance Dashboard:** Theo dõi lưu lượng truy cập (RPS), tỷ lệ lỗi 4xx/5xx, và thời gian phản hồi trung bình của từng API.
2. **Integration Health Dashboard:** Giám sát trạng thái các tác vụ đồng bộ dữ liệu (Sync Jobs), số lượng bản ghi được thêm mới/cập nhật và các lỗi xảy ra trong quá trình ETL.
3. **Infrastructure Dashboard:** Theo dõi sức khỏe của các Container và Database.

5.4.2 Cơ chế Cảnh báo (Alerting)

Dựa trên tính năng *Kibana Alerting*, hệ thống thiết lập các quy tắc (rules) để gửi thông báo tự động qua Email hoặc Webhook (Slack/Telegram) khi xảy ra sự cố:

Mức độ	**Điều kiện kích hoạt**	**Hành động**
Critical	Tỷ lệ lỗi 5xx > 5% trong 1 phút	Gửi SMS, Email Admin
Critical	Database Connection Failed	Gửi SMS, Email Admin
Warning	CPU Usage > 85% trong 5 phút	Gửi Email cảnh báo
Warning	Phát hiện Slow Query > 1s	Log lại để tối ưu sau
Info	Job đồng bộ dữ liệu hoàn tất	Gửi báo cáo qua Slack

Bảng 5.4.1: Ma trận quy tắc cảnh báo

Chương 6

Data Governance và Quy trình Tích hợp liên tục (CI/CD)

6.1 Quản trị Dữ liệu (Data Governance) và Chất lượng Dữ liệu (Data Quality)

Trong kiến trúc trục tích hợp dữ liệu của trường đại học, tính đa dạng của các nguồn dữ liệu (SIS, LMS, HRM, Tài chính) đặt ra thách thức lớn về tính nhất quán. Do đó, việc thiết lập cơ chế quản trị dữ liệu (Data Governance) song hành với các giải pháp kỹ thuật là yêu cầu bắt buộc.

Hệ thống áp dụng ba trụ cột quản trị chính:

6.1.1 Quản trị Dữ liệu (Data Governance)

6.1.1.1 Quản lý Siêu dữ liệu (Metadata Management)

Để đảm bảo sự thấu hiểu chung về dữ liệu giữa các phòng ban, hệ thống xây dựng bộ từ điển dữ liệu (Data Dictionary) chuẩn. Metadata bao gồm định nghĩa trường, kiểu dữ liệu, các danh mục mã (code list) và mối quan hệ thực thể. Metadata đóng vai trò là "ngôn ngữ chung", giúp giảm thiểu xung đột ngữ nghĩa khi tích hợp.

6.1.1.2 Quyền sở hữu dữ liệu (Data Ownership & Stewardship)

Trách nhiệm về dữ liệu được phân định rõ ràng theo từng miền (domain):

- **Data Owner:** Chịu trách nhiệm pháp lý và phê duyệt các chính sách chất lượng dữ liệu.

- **Data Steward:** Chịu trách nhiệm vận hành, rà soát và xử lý các vấn đề dữ liệu ở mức kỹ thuật.

6.1.1.3 Kiểm soát truy cập (Data Access Control)

Hệ thống tuân thủ nguyên tắc đặc quyền tối thiểu (Least Privilege). Tầng cơ sở dữ liệu áp dụng các chính sách phân quyền (`GRANT/REVOKE`), trong khi tầng ứng dụng và API Gateway kiểm soát quyền truy cập thông qua cơ chế RBAC và xác thực bằng JWT.

6.1.2 Kiểm soát Chất lượng Dữ liệu (Data Quality - DQ)

Mọi luồng dữ liệu khi đi qua Data Integration Server đều phải vượt qua các cổng kiểm soát chất lượng (Quality Gates) nghiêm ngặt, đảm bảo tính nhất quán theo tiêu chuẩn [3]:

- **Tính hợp lệ (Validity Checks):** Kiểm tra định dạng (Regex cho email, MSSV), ràng buộc miền giá trị và tính toàn vẹn (null/not-null).
- **Tính nhất quán (Consistency Checks):** Đảm bảo dữ liệu quan hệ khớp với metadata (ví dụ: Mã Khoa trong hồ sơ sinh viên phải tồn tại trong danh mục Khoa).
- **Tính kịp thời (Timeliness Checks):** Phát hiện và đánh dấu các bản ghi trễ hạn (late-arriving data) để có quy trình xử lý riêng.
- **Phát hiện trùng lặp (Duplicate Detection):** Sử dụng thuật toán so khớp trên các trường định danh (CCCD, Email, MSSV) để loại bỏ dữ liệu thừa.

6.1.3 Cơ chế xử lý lỗi dữ liệu

Dữ liệu không đạt chuẩn sẽ không bị loại bỏ hoàn toàn mà được chuyển hướng vào hàng đợi lỗi (*Error Queue/Dead Letter Queue*). Sự kiện này được ghi log chi tiết vào Elasticsearch và gửi cảnh báo đến Data Steward để xử lý thủ công, đảm bảo không làm gián đoạn luồng dữ liệu chính.

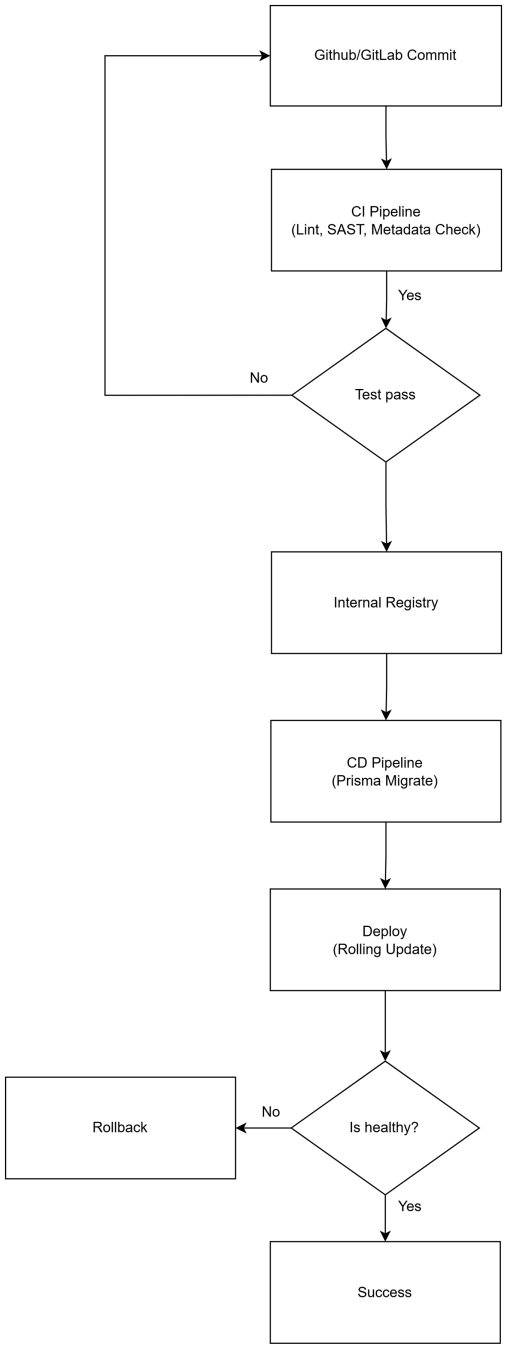
6.2 Tích hợp Data Governance vào CI/CD

Để đảm bảo tính liên tục và an toàn khi triển khai, quy trình CI/CD (Continuous Integration/Continuous Deployment) được mở rộng với các bước kiểm thử dữ liệu chuyên sâu.

6.2.1 Quy trình triển khai

Quy trình tự động hóa bao gồm các bước sau:

1. **Source Code Management:** Developer commit mã nguồn lên GitLab.
2. **Continuous Integration (CI):** Pipeline tự động thực thi:
   - Unit Test và Linting cho mã nguồn.
   - Kiểm thử bảo mật tĩnh (SAST).
   - **Metadata Consistency Checks:** So sánh schema trong mã nguồn với metadata hiện tại để phát hiện thay đổi phá vỡ (breaking changes).
3. **Build Artifact:** Đóng gói Docker Image và đẩy lên Registry nội bộ.
4. **Database Migration:** Sử dụng **Prisma Migration** để cập nhật cấu trúc CSDL, bao gồm kiểm tra ràng buộc khóa ngoại và tương thích ngược.
5. **Continuous Deployment (CD):** Triển khai bản cập nhật theo chiến lược **Rolling Update** để đảm bảo tính sẵn sàng (Zero-downtime).
6. **Observability:** Hệ thống giám sát (Elasticsearch + Grafana) theo dõi sức khỏe ứng dụng sau triển khai. Nếu phát hiện tỷ lệ lỗi tăng đột biến, pipeline tự động kích hoạt quy trình Rollback.



Hình 6.2.1: Quy trình CI/CD tích hợp kiểm tra dữ liệu và giám sát

Chương 7

Chiến lược Sao lưu và Phục hồi Dữ liệu (Backup Strategy)

Nhóm thực hiện đề xuất giải pháp sao lưu sử dụng **pgBackRest** kết hợp với hệ thống lưu trữ đối tượng **MinIO** triển khai nội bộ. Giải pháp này đảm bảo tính bảo mật dữ liệu của trường đại học trong khi vẫn tận dụng được sự linh hoạt của kiến trúc S3.

Bảng 7.0.1: So sánh MinIO với các giải pháp lưu trữ đám mây

Tiêu chí	**MinIO (On-premise)**	**AWS S3**	**Google Cloud Storage**
Mô hình triển khai	Private Cloud (Nội bộ)	Public Cloud	Public Cloud
Vị trí dữ liệu	Mạng nội bộ nhà trường	Datacenter nhà cung cấp	Datacenter nhà cung cấp
Khả năng tích hợp	Tương thích hoàn toàn S3 API	Native	SDK riêng
Chi phí vận hành	Thấp (Tận dụng phần cứng sẵn có)	Cao (Theo dung lượng)	Cao (Theo dung lượng)
Độ trễ	Rất thấp (LAN)	Trung bình	Trung bình
Độ phù hợp	**Rất cao**	Thấp	Thấp

7.1 Chiến lược và Chu kỳ sao lưu

Để cân bằng giữa hiệu năng hệ thống và an toàn dữ liệu, hệ thống áp dụng chiến lược sao lưu hỗn hợp:

- **Full Backup (Toàn phần):** Thực hiện **1 lần/tuần** vào thời điểm thấp điểm (00:00 – 03:00 Chủ nhật). Bản sao lưu này chứa toàn bộ dữ liệu hệ thống.
- **Incremental Backup (Gia tăng):** Thực hiện **hằng ngày** vào lúc 01:00 sáng. Bản sao lưu này chỉ chứa các dữ liệu thay đổi so với lần backup trước đó, giúp tiết kiệm dung lượng và thời gian.
- **Lưu trữ:** Các bản sao lưu được giữ lại ít nhất 1 tháng (Retention Policy).

7.2 Phương thức kỹ thuật

Hệ thống sử dụng công cụ **pgBackRest** nhờ các ưu điểm vượt trội về hiệu năng và khả năng tích hợp S3. Quy trình xử lý file backup trước khi đẩy lên MinIO bao gồm:

1. **Nén dữ liệu:** Sử dụng thuật toán Gzip/LZ4 để giảm dung lượng đường truyền và lưu trữ.
2. **Mã hóa:** Áp dụng mã hóa AES-256 để đảm bảo dữ liệu không thể bị đọc trộm ngay cả khi hệ thống lưu trữ bị xâm nhập.

7.3 Cam kết chỉ số phục hồi (RPO & RTO)

Dựa trên chiến lược sao lưu toàn phần kết hợp sao lưu gia tăng hằng ngày, hệ thống cam kết:

- **Recovery Point Objective (RPO): 1 ngày.** Trong trường hợp xấu nhất, lượng dữ liệu tối đa bị mất tương đương với các giao dịch phát sinh từ sau lần Incremental Backup gần nhất (tối đa 24 giờ).
- **Recovery Time Objective (RTO): 1–3 giờ.** Thời gian cần thiết để tải các bản backup từ MinIO, giải mã, giải nén và khôi phục vào hệ quản trị cơ sở dữ liệu.

Chương 8

Kế hoạch thực hiện đồ án tốt nghiệp

8.1 Mục tiêu tổng thể

Đồ án tốt nghiệp nhằm hoàn thiện và triển khai hệ thống Trục Tích Hợp Dữ Liệu cho Trường Đại Học dựa trên nền tảng kiến trúc API-led Connectivity, kế thừa kết quả thiết kế kiến trúc từ đồ án chuyên ngành, tiến tới hiện thực hóa hệ thống ở mức độ thực tế (production-like).

Mục tiêu chính:

- Xây dựng trục tích hợp dữ liệu hoạt động theo mô hình One-way Integration Hub
- Đảm bảo tính nhất quán, bảo mật và khả năng mở rộng
- Hoàn thiện tài liệu kỹ thuật, triển khai, kiểm thử và đánh giá hệ thống

8.2 Phân chia giai đoạn thực hiện

Dự án được dự định chia thành 6 giai đoạn chính, triển khai liên tục từ tháng 01/2026 đến tháng 05/2026.

8.2.1 Giai đoạn 1: Phân tích & Hoàn thiện thiết kế (01/2026)

Mục tiêu: Rà soát toàn bộ kiến trúc đã đề xuất trong đồ án chuyên . Hoàn thiện thiết kế chi tiết cho đồ án tốt . Chuẩn hóa yêu cầu chức năng và phi chức năng

Công việc thực hiện:

- Phân tích lại yêu cầu hệ thống trục tích hợp

- Hoàn thiện: Deployment Architecture; Security Architecture (4 layer security); Data Flow Diagram; API-led Connectivity Model
- Thiết kế: Data Model trung tâm (PostgreSQL); API Contracts (OpenAPI); Quy trình backup, logging, monitoring
- Lập kế hoạch triển khai chi tiết cho các giai đoạn tiếp theo

Phân công

- Tuấn: Thiết kế kiến trúc tổng thể; Security Architecture; API Gateway & Auth Flow
- Huy: Data Model & Database Design; Backup, Data Governance; Observability & Logging Design

Sản phẩm đầu ra

- Tài liệu thiết kế kiến trúc hoàn chỉnh
- Sơ đồ hệ thống (PNG)
- Chương Kiến trúc & Thiết kế trong báo cáo

8.2.2 Giai đoạn 2: Phát triển Core Services (02/2026)

Mục tiêu: Hiện thực hóa trục tích hợp dữ liệu; Xây dựng các API chính của hệ thống
Công việc thực hiện

- Phát triển: Data Integration Server (ingest data từ SIS/LMS giả lập); Data Sharing Server (API truy vấn dữ liệu)
- Xây dựng: API chuẩn hóa dữ liệu; Validation & Mapping
- Áp dụng: JWT Authentication; Role/Permission-based Authorization
- Ghi log nghiệp vụ (Security Logging)

Phân công

- Tuấn: Auth & Authorization; API Gateway Integration; Security Middleware
- Huy: Data Processing Logic; Data Validation; Database Interaction

Sản phẩm đầu ra

- Trục tích hợp dữ liệu hoạt động
- API truy vấn dữ liệu ổn định
- Swagger/OpenAPI Documentation

8.2.3 Giai đoạn 3: Xây dựng hạ tầng & DevOps (03/2026)

Mục tiêu: Chuẩn bị môi trường triển khai và pipeline DevOps; Đảm bảo hệ thống có thể build – deploy – rollback

Công việc thực hiện

- Thiết lập:Git repository (GitHub/GitLab); CI/CD Pipeline
- Docker hóa: NestJS Services; Auth Service
- Cấu hình: Kong API Gateway; PostgreSQL; MinIO
- Thiết lập: Prisma ORM & Prisma Migrate. Biến môi trường & Secret Management
- Triển khai pipeline: build image; migrate schema; deploy rolling/canary

Phân công

- Tuấn: CI/CD Pipeline; Kong Gateway; Auth Service.
- Huy: Docker & PostgreSQL; Prisma & Migration; MinIO Backup Infrastructure

Sản phẩm đầu ra

- Pipeline CI/CD hoạt động
- Môi trường triển khai hoàn chỉnh
- Tài liệu DevOps & Deployment

8.2.4 Giai đoạn 4: Hoàn thiện bảo mật, logging & monitoring (04/2026)

Mục tiêu: Đảm bảo hệ thống đạt mức bảo mật và giám sát cao; Chuẩn bị cho kiểm thử tổng thể

Công việc thực hiện:

- Tích hợp: Filebeat / Logstash; Elasticsearch; Kibana Dashboard
- Cấu hình: Alerting; Rate Limiting; Input Validation
- Hoàn thiện: Backup & Restore; RPO/RTO
- Kiểm thử bảo mật: JWT; Permission; Rate limiting

Phân công

- Tuấn: Security Testing; API Gateway Policies
- Huy: Logging Pipeline; Monitoring Dashboard; Backup & Restore Testing

Sản phẩm đầu ra

- Hệ thống giám sát hoạt động
- Dashboard Kibana
- Báo cáo bảo mật & logging

8.2.5 Giai đoạn 5: Kiểm thử tổng thể & tối ưu (05/2026 - đầu tháng)

Mục tiêu: Đánh giá chất lượng hệ thống; Tối ưu hiệu năng và độ ổn định

Công việc thực hiện

- Thực hiện: Unit Test; Integration Test; Load Test (giả lập LMS/MyBK)
- Đánh giá: Hiệu năng API; Khả năng chịu tải
- Tối ưu: Query; Index DB; Cache (nếu cần)

Sản phẩm đầu ra

- Báo cáo kiểm thử
- Biểu đồ đánh giá hiệu năng
- Phiên bản hệ thống ổn định

8.2.6 Giai đoạn 6: Hoàn thiện báo cáo & bảo vệ (Cuối 05/2026)

Mục tiêu: Hoàn tất đồ án tốt nghiệp; Chuẩn bị bảo vệ
Công việc thực hiện

- Viết và chỉnh sửa: Báo cáo đồ án; Phụ lục kỹ thuật
- Chuẩn bị: Slide thuyết trình; Demo hệ thống
- Tổng hợp: Kết luận; Hướng phát triển tương lai

Sản phẩm đầu ra

- Báo cáo hoàn chỉnh
- Slide bảo vệ
- Hệ thống demo

Chương 9

Tổng kết đồ án chuyên ngành

9.1 Những điểm đã đạt được

Trong khuôn khổ đồ án chuyên ngành, nhóm đã tiến hành nghiên cứu, phân tích và đề xuất một giải pháp tổng thể cho bài toán tích hợp dữ liệu trong môi trường trường đại học, nơi tồn tại nhiều hệ thống thông tin độc lập như hệ thống quản lý đào tạo, hệ thống quản lý học tập (LMS), hệ thống nhân sự, tài chính và các cổng thông tin nội bộ. Thông qua quá trình khảo sát thực tế và tham khảo các mô hình tích hợp hiện đại, đồ án đã xác định rõ nhu cầu cấp thiết về một trục tích hợp dữ liệu đóng vai trò trung gian, giúp chuẩn hóa, thu thập và cung cấp dữ liệu một cách nhất quán giữa các hệ thống.

Kết quả nổi bật đầu tiên của đồ án là việc xây dựng được cái nhìn tổng quan và có hệ thống về các mô hình trục tích hợp dữ liệu, bao gồm kiến trúc ESB truyền thống và các kiến trúc hiện đại hơn như API-led Connectivity. Trên cơ sở so sánh ưu, nhược điểm của từng mô hình, đồ án đã đề xuất kiến trúc trục tích hợp dữ liệu dựa trên API-led Connectivity, phù hợp với xu hướng microservices và yêu cầu mở rộng linh hoạt trong môi trường đại học. Kiến trúc đề xuất đã làm rõ vai trò của các thành phần chính như API Gateway, Integration Server, Data Sharing Server, Auth Service và cơ sở dữ liệu trung tâm, cũng như luồng dữ liệu một chiều từ các hệ thống nguồn về trục tích hợp.

Bên cạnh kiến trúc tổng thể, đồ án chuyên ngành cũng đạt được những kết quả quan trọng trong việc thiết kế các khía cạnh kỹ thuật cốt lõi của hệ thống. Cụ thể, nhóm đã đề xuất nền tảng công nghệ phù hợp cho từng thành phần, trong đó NestJS được lựa chọn làm nền tảng backend cho các dịch vụ tích hợp và chia sẻ dữ liệu, PostgreSQL được sử dụng làm cơ sở dữ liệu trung tâm, Kong API Gateway đóng vai trò tuyến phòng thủ và điều phối truy cập, cùng với MinIO cho lưu trữ backup. Ngoài ra, đồ án đã xây dựng được mô hình bảo mật tổng thể nhiều lớp, bao gồm xác thực và phân quyền dựa trên

JWT, mã hóa dữ liệu khi truyền và khi lưu trữ, cũng như các cơ chế giám sát, ghi log và cảnh báo nhằm đảm bảo an toàn và khả năng truy vết của hệ thống.

Một đóng góp đáng kể khác của đồ án là việc thiết kế chi tiết các sơ đồ kiến trúc, sơ đồ luồng dữ liệu và sơ đồ triển khai, giúp làm rõ cách thức các thành phần tương tác với nhau trong một hệ thống trục tích hợp hoàn chỉnh. Những sơ đồ này không chỉ hỗ trợ cho việc hiểu rõ giải pháp đề xuất mà còn đóng vai trò là nền tảng quan trọng để hiện thực hóa hệ thống trong giai đoạn đồ án tốt nghiệp tiếp theo. Ngoài ra, đồ án cũng đã đề cập đến các yếu tố vận hành như chiến lược sao lưu dữ liệu, chỉ số phục hồi sau sự cố (RPO, RTO), cũng như định hướng triển khai DevOps và CI/CD, cho thấy tính thực tiễn và khả năng áp dụng của giải pháp trong môi trường thực tế.

9.2 Những điểm cần cải thiện

Tuy đạt được nhiều kết quả tích cực, đồ án chuyên ngành vẫn còn tồn tại một số hạn chế nhất định. Do giới hạn về thời gian và phạm vi của đồ án, hệ thống mới chỉ dừng lại ở mức thiết kế kiến trúc và định hướng công nghệ, chưa thể triển khai và kiểm chứng toàn bộ các thành phần trong môi trường thực tế. Các vấn đề phức tạp như đồng bộ dữ liệu theo thời gian thực, xử lý xung đột dữ liệu giữa các hệ thống nguồn, tối ưu hiệu năng khi lưu lượng truy cập lớn hay đánh giá mức độ chịu tải của hệ thống vẫn chưa được thực nghiệm đầy đủ. Bên cạnh đó, việc tích hợp trực tiếp với các hệ thống thực tế của trường đại học cũng gặp nhiều khó khăn do hạn chế về quyền truy cập và dữ liệu mẫu, khiến quá trình đánh giá chỉ có thể dựa trên các kịch bản giả lập.

Ngoài ra, đồ án cũng nhận thấy rằng việc thiết kế một trục tích hợp dữ liệu đòi hỏi sự phối hợp chặt chẽ giữa nhiều yếu tố như quản trị dữ liệu, chuẩn hóa dữ liệu và quản lý thay đổi hệ thống. Những khía cạnh này mới chỉ được đề cập ở mức định hướng và cần được nghiên cứu sâu hơn, đặc biệt là các vấn đề liên quan đến Data Governance, Data Quality và quy trình vận hành lâu dài của hệ thống. Đây là những nội dung quan trọng cần được tiếp tục hoàn thiện trong giai đoạn phát triển tiếp theo để đảm bảo hệ thống trục tích hợp không chỉ hoạt động đúng về mặt kỹ thuật mà còn bền vững về mặt quản lý và vận hành.

9.3 Kết luận chung

Tổng kết lại, đồ án chuyên ngành đã hoàn thành tốt mục tiêu đề ra khi xây dựng được một nền tảng lý thuyết và thiết kế kiến trúc tương đối toàn diện cho hệ thống trục tích

hợp dữ liệu trong trường đại học. Những kết quả đạt được không chỉ có giá trị học thuật mà còn mang tính thực tiễn cao, tạo tiền đề vững chắc cho việc triển khai và hoàn thiện hệ thống trong đồ án tốt nghiệp. Các hạn chế và khó khăn được nhận diện trong quá trình thực hiện cũng chính là động lực và định hướng rõ ràng cho giai đoạn tiếp theo, nơi nhóm sẽ tập trung vào hiện thực hóa, kiểm thử và đánh giá hệ thống trong môi trường gần với thực tế hơn.

Danh mục tham khảo

[1] Amazon Web Services. What is data integration? https://aws.amazon.com/what-is/data-integration/, 2025. Truy cập ngày: 23-10-2025.

[2] Confluent. What is publish/subscribe messaging? https://www.confluent.io/learn/publish-subscribe/, 2025. Truy cập ngày: 23-10-2025.

[3] Gartner. Definition of data consistency. https://www.gartner.com/en/information-technology/glossary/data-consistency, 2025. Truy cập ngày: 23-10-2025.

[4] Jiawei Han, Jian Pei, and Hanghang Tong. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 4th edition, 2023.

[5] IBM. Data integration: An overview. https://www.ibm.com/topics/data-integration, 2025. Truy cập ngày: 23-10-2025.

[6] Informatica. What is a data integration hub? https://www.informatica.com/resources/articles/what-is-a-data-integration-hub.html, 2025. Truy cập ngày: 23-10-2025.

[7] Microsoft. Online transaction processing (oltp). https://learn.microsoft.com/en-us/azure/architecture/data-guide/relational-data/online-transaction-processing, 2025. Truy cập ngày: 23-10-2025.

[8] Microsoft. Publisher-subscriber pattern. https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber, 2025. Truy cập ngày: 23-10-2025.

[9] Deepak Pareek. *Business Intelligence for Telecommunications*. CRC Press, 2007.

[10] Salesorce. What is api-led connectivity? unlock business agility. https://www.salesforce.com/ap/blog/api-led-connectivity/, 2025. Truy cập ngày: 23-10-2025.

[11] Daizhong Su and Shuyan Ji. Integrated worm design based on distributed heterogeneous system. In *Proceedings of the International Conference on Distributed Systems*, pages 36–42, 2006.

[12] Wikipedia. Enterprise service bus. https://en.wikipedia.org/wiki/Enterprise_service_bus, 2025. Truy cập ngày: 23-10-2025.

[13] Wikipedia. Extract, transform, load. https://en.wikipedia.org/wiki/Extract,_transform,_load, 2025. Truy cập ngày: 23-10-2025.